

МЕТОДИЧКА ДЛЯ САМОСТОЯТЕЛЬНОГО ИЗУЧЕНИЯ

FULL-STACK РАЗРАБОТКА

От устройства веба и интерфейса
до серверной логики, архитектуры
и надёжной работы приложения

39 глав • 16 страниц со схемами

Сквозной проект: магазин с заказами

Теория, примеры кода и визуальные объяснения

Автор: Карина Куслина · @Miramisha

Методичка создана с использованием искусственного интеллекта (ИИ).

01

ИНТЕРФЕЙС

02

СЕРВЕР

03

ДАННЫЕ

TLS / SSH / API / SQL / SECURITY / ARCHITECTURE

Издание со схемами • 25 сентября 2026

Оглавление

Читай последовательно или выбирай главу. Названия в оглавлении и PDF-закладки открывают нужный раздел.

Как пользоваться методичкой	4
01. Картина целиком: браузер, сервер и данные	5
02. Структуры данных, алгоритмы и модель выполнения	7
03. Терминал, Git и рабочая среда	8
04. IP, TCP, UDP, DNS и диагностика сети	9
05. TLS, HTTPS и SSH	10
06. HTTP, cookies и кеш браузера	12
07. HTML, CSS и доступность	13
08. JavaScript, DOM и асинхронность	14
09. TypeScript и границы доверия	15
10. Компоненты, состояние и архитектура frontend	16
11. CSR, SSR, SSG, сборка и производительность UI	18
12. Фреймворки: карта и критерии выбора	19
13. Внутри сервера: процессы, память и event loop	21
14. Жизненный цикл запроса и middleware	23
15. Бизнес-логика и состояния заказа	25
16. Проектирование API и контрактов	27
17. Идемпотентность и неопределённый результат	28
18. SQL, модель данных и ORM	30
19. Транзакции, изоляция и конкурентность	32
20. Индексы, запросы, миграции и восстановление	34

21. Аутентификация, авторизация и сессии	35
22. Безопасность входов, браузера и файлов	37
23. Типы ошибок и стратегия обработки	38
24. SOLID, DRY, KISS, YAGNI и чистый код	40
25. Модули, Clean Architecture, Hexagonal и DI	41
26. DDD, монолит, микросервисы и события	43
27. Внешние API, платежи и webhooks	44
28. Очереди, Outbox и Saga	45
29. Таймауты, retry, circuit breaker и backpressure	47
30. Кеширование, CDN и инвалидация	48
31. WebSocket, SSE и обновления в реальном времени	50
32. Тестирование, отладка и проверка контрактов	51
33. Linux, Docker и управление процессом	52
34. Reverse проху, CI/CD и выпуск изменений	53
35. Логи, метрики, трассировка и инциденты	55
36. Производительность и масштабирование	56
37. Командные правила и архитектурные решения	57
38. Сквозной разбор: безопасное создание заказа	58
39. Учебный маршрут и итоговый проект	60
Приложение А. Ответы для самопроверки	61
Приложение В. Короткий словарь	63
Приложение С. Официальные источники	64

Как пользоваться методичкой

Автор: **Карина Куслина** · @Miramisha

Методичка создана с использованием искусственного интеллекта (ИИ).

Эта методичка объединяет темы full-stack разработки в учебный маршрут: основы веба, frontend, backend, сети, архитектура, данные, ошибки, безопасность и эксплуатация. Задача - научиться объяснять работу приложения и принимать технические решения, а не только воспроизводить команды из урока. Предполагается, что ты начинаешь с базового уровня; сложные темы вводятся через знакомые сценарии.

Сквозной пример - небольшой интернет-магазин. Пользователь смотрит каталог, добавляет товары, оформляет заказ и видит его статус. На этом примере удобно обсуждать интерфейс, права доступа, транзакции, оплату, очереди и сбои. Деньги в примерах хранятся в минимальных единицах валюты, например в копейках; клиент не назначает цену заказа.

Основные языки примеров - JavaScript/TypeScript и SQL PostgreSQL. Это учебный выбор для связности материала, а не требование использовать именно такой стек. Фрагменты с пометкой «схема» или «псевдокод» показывают устройство решения: они не составляют готовое приложение. В остальных фрагментах тоже могут быть опущены окружающий проект, импорты и настройка запуска; условия указаны рядом.

Как заниматься

Прочитай объяснение, перескажи его своими словами, разбери пример и выполни практику. После этого ответь на вопрос самопроверки без подсказки. Не переходи к распределённым транзакциям, пока не можешь объяснить обычную транзакцию и повтор HTTP-запроса. Возвращаться к предыдущим главам по мере работы над проектом нормально.

В каждой главе есть теория, пример, типичная ошибка и небольшое упражнение. В конце собраны проектные этапы, ответы для самопроверки, словарь и официальные источники. В это издание добавлены 16 страниц со схемами рядом с соответствующими главами. У каждой схемы есть пояснение и вопрос. Оглавление и PDF-закладки помогают возвращаться к нужной теме; схемы доступны как вложенные закладки.

Уровни освоения

Первый уровень - объяснить понятие на бытовом примере. Второй - реализовать небольшой сценарий. Третий - предсказать сбой и проверить восстановление. Например, «знаю SSH» означает не только умею подключиться, но и понимаю различие ключа пользователя и ключа сервера, а также причину предупреждения об изменённом ключе хоста.

Сведения о концепциях отделены от версий инструментов. Команды создания проектов, требования к среде и конкретные параметры безопасности сверяй с документацией выбранной версии. Ссылки на первоисточники приведены в конце; дата подготовки методички - 25 сентября 2026 года.

ГЛАВА 01

01. Картина целиком: браузер, сервер и данные

Три зоны ответственности

Frontend отображает состояние системы и принимает действия пользователя. Backend проверяет полномочия и бизнес-правила, обращается к данным и внешним сервисам. База данных хранит информацию и обеспечивает ограничения целостности. Даже если все три части запущены на одном ноутбуке, их роли различаются.

Браузер нельзя считать доверенной средой: пользователь может изменить JavaScript, подменить запрос или отправить его через curl. Поэтому проверка «количество положительное» полезна в интерфейсе для удобства, но обязательна и на сервере. Цена товара, роль пользователя и владелец заказа определяются доверенными серверными данными.

Слово «сервер» обозначает и машину, и программу. На одной машине могут работать reverse proxy, backend и база. Одно приложение, наоборот, может быть распределено по нескольким машинам. Для обсуждения архитектуры уточняй, идёт ли речь о процессе, сервисе или физическом узле.

Что происходит при открытии страницы

Браузер разбирает URL: схему, домен, порт, путь и query-параметры. Затем получает адрес через DNS, устанавливает транспортное соединение и при HTTPS защищённый канал. Сервер возвращает документ; браузер загружает ресурсы, строит DOM и применяет CSS. JavaScript может запросить JSON и обновить интерфейс без полной перезагрузки.

```
Браузер -> DNS: какой адрес у shop.example?  
Браузер -> reverse proxy: HTTPS GET /catalog  
Прокси -> backend: запрос к приложению  
Backend -> БД: получить товары  
БД -> backend -> браузер: данные и ответ
```

Это концептуальная схема, а не обязательная последовательность отдельных сетевых обменов: DNS и ресурсы могут быть в кеше, соединение - уже открыто, а HTML - подготовлен заранее.

Как читать требования

Функциональное требование описывает действие: «покупатель отменяет неоплаченный заказ». Нефункциональное - качество или ограничение: «операция доступна только владельцу», «данные сохраняются после перезапуска», «95% ответов укладываются в согласованный бюджет времени». Хорошее требование можно проверить.

Типичная ошибка. Начать с выбора микросервисов, не описав сценарии и данные. Архитектура должна объяснять, как выполнить требования и с какими издержками.

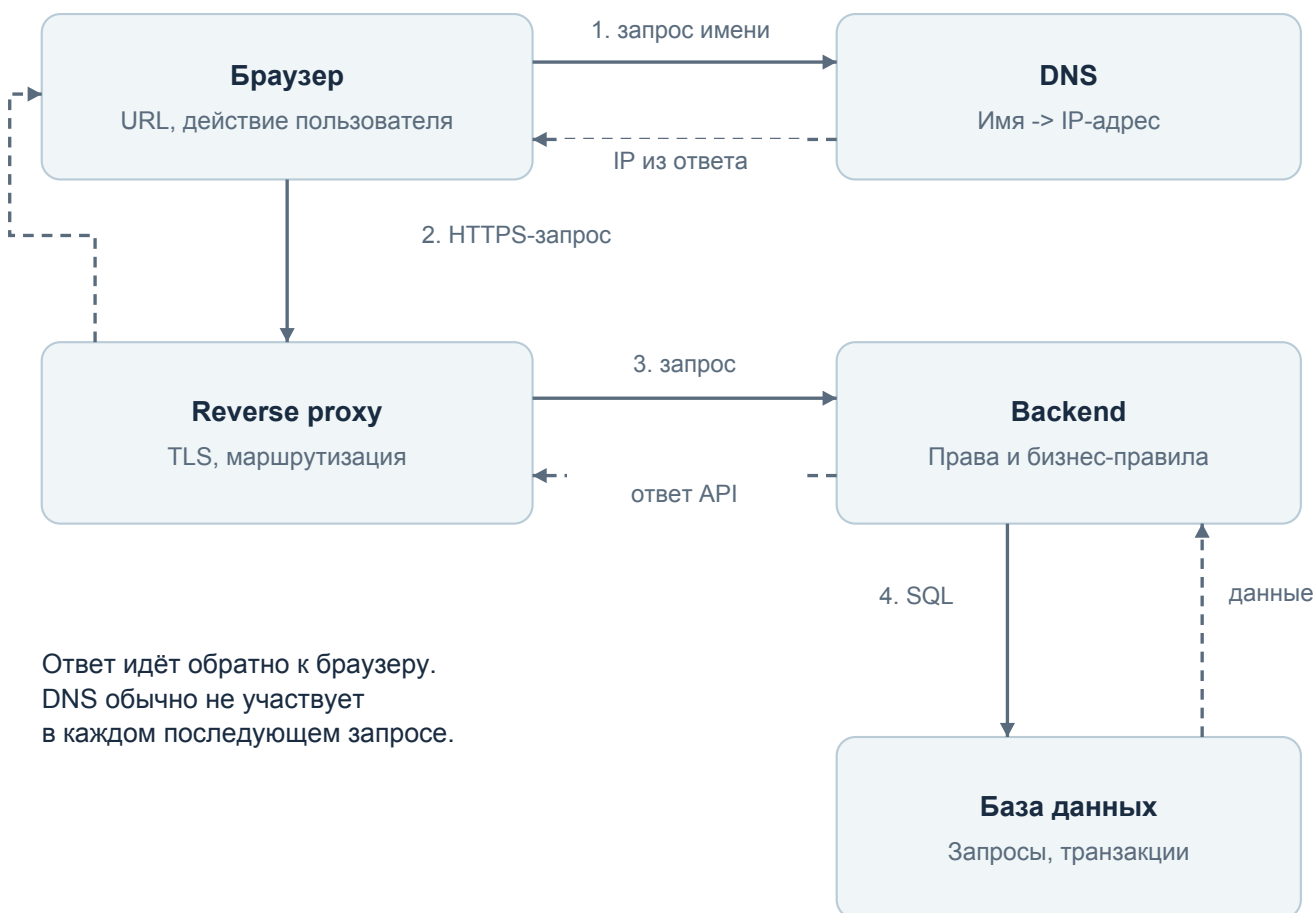
Практика. Нарисуй путь оформления заказа и подпиши, где проверяются цена, остаток, пользователь и статус.

Самопроверка. Почему отключённая кнопка «Удалить» не защищает чужой заказ?

СХЕМА 01 / К ГЛАВЕ 01

Путь запроса через систему

Стрелки показывают обращения между участниками; пунктир - возврат данных. DNS помогает найти адрес, а бизнес-решение принимает backend.



Как читать схему

Проследи запрос создания заказа от браузера до БД. Затем найди обратный путь ответа и отдельно объясни роль DNS.

Проверь себя. Если API вернул HTTP 500, какие этапы уже точно были пройдены?

ГЛАВА 02

02. Структуры данных, алгоритмы и модель выполнения

Что хранить и как искать

Массив удобен для последовательности: список строк заказа, очередь отображаемых карточек. Map хранит соответствие ключа и значения: товар по идентификатору. Set представляет уникальное множество: выбранные категории. Стек обрабатывает последний добавленный элемент первым, очередь - первый добавленный первым. Дерево описывает иерархию, граф - произвольные связи.

Сложность оценивает рост затрат при увеличении входа. Один проход по n строкам - $O(n)$; два вложенных прохода по тем же n элементам - $O(n^2)$. Поиск по хеш-таблице в среднем близок к $O(1)$, но не является безусловной гарантией для любого случая. Бинарный поиск имеет $O(\log n)$, если данные отсортированы и доступ к середине эффективен.

В веб-приложении важны не только вычисления. Алгоритм из ста быстрых шагов может проиграть одному сетевому запросу. Сначала измеряй, где время тратится: CPU, сеть, ожидание соединения или база.

Пример: соединить товары и строки заказа

Повторный find по всему каталогу для каждой строки создаёт лишнюю работу. Индексирование один раз делает намерение понятнее и сокращает число просмотров.

```
const products = [
  { id: 'p1', title: 'Книга' },
  { id: 'p2', title: 'Блокнот' }
];
const items = [{ productId: 'p2', quantity: 2 }];
const byId = new Map(products.map(p => [p.id, p]));
const result = items.map(item => ({
  ...item,
  title: byId.get(item.productId)?.title ?? 'Не найден'
}));
```

Построение индекса занимает $O(m)$ времени и дополнительную память, обработка строк - $O(n)$ в среднем. Это разумный обмен памяти на скорость, если действительно нужны множественные поиски.

Значения, ссылки и рекурсия

Изменение объекта через одну ссылку видно другим владельцам той же ссылки. Поверхностная копия через spread не копирует вложенные объекты. Рекурсия полезна для деревьев, но нуждается в условии остановки; слишком глубокий вызов может исчерпать стек. Цикл часто проще для линейного обхода.

Типичная ошибка. Оптимизировать арифметику и не замечать тысячу запросов к базе внутри цикла.

Практика. Удали дубликаты идентификаторов через Set и объясни, почему это не решает проблему уникальности записей при двух одновременных запросах к серверу.

Самопроверка. Чем индекс в памяти отличается от ограничения UNIQUE в базе?

ГЛАВА 03

03. Терминал, Git и рабочая среда

Командная строка как интерфейс

Терминал позволяет воспроизводимо запускать программу, тесты и диагностику. Текущий каталог влияет на относительные пути. Процесс получает аргументы, переменные окружения и потоки stdin, stdout, stderr. Нулевой код завершения обычно означает успех, ненулевой - ошибку, которую может обработать CI.

Pipe передаёт вывод одной команды на вход другой. Перенаправление в файл удобно для отчётов, но символ > перезаписывает файл. Кавычки нужны для путей с пробелами; пользовательский ввод нельзя бездумно вставлять в команды оболочки. Это такой же риск инъекции, как сборка SQL строкой.

```
pwd
ls
mkdir fullstack-study
cd fullstack-study
git init
git status
```

Команды рассчитаны на Unix-подобную оболочку. На Windows аналогичный опыт можно получить в WSL или адаптировать команды к PowerShell.

Как устроен Git

Рабочая директория содержит текущие файлы, индекс - подготовленные изменения, коммит - снимок состояния с метаданными. Ветка указывает на коммит. Коммит локален; push отправляет историю в удалённый репозиторий. Pull request - механизм обсуждения и проверки изменения на платформе, а не отдельный тип объекта Git.

```
git switch -c feature/catalog
git add src/catalog.ts
git diff --staged
git commit -m "Add catalog filtering"
```

Merge объединяет истории; rebase переносит коммиты на другую основу. Переписывание уже опубликованной общей истории требует согласования с командой. При конфликте нужно понять смысл двух изменений, отредактировать итог и проверить поведение; механический выбор одной стороны может потерять нужный код.

Зависимости и воспроизводимость

Менеджер пакетов устанавливает библиотеки. Манифест описывает прямые зависимости, lock-файл фиксирует разрешённое дерево версий. Совместная фиксация манифеста и lock-файла уменьшает различия между средами. Разделяй конфигурацию и секреты: пример переменных можно хранить в репозитории, реальные пароли - нет. Если секрет попал в историю, одного удаления файла недостаточно: секрет нужно отозвать.

Типичная ошибка. Коммитить огромный набор несвязанных изменений: его трудно проверить, отменить и обсуждать.

Практика. Создай две ветки, измени одну строку по-разному, объедини их и осмысленно разреши конфликт.

Самопроверка. Почему успешный запуск на твоём компьютере не гарантирует успешную сборку в CI?

ГЛАВА 04

04. IP, TCP, UDP, DNS и диагностика сети

Уровни взаимодействия

IP доставляет пакеты между адресами; порт помогает направить данные нужному транспортному endpoint. TCP предоставляет упорядоченный поток байтов с обнаружением потерь и повторной передачей. Приложение само определяет границы своих сообщений в этом потоке. UDP передаёт отдельные датаграммы без встроенной гарантии доставки и порядка; над ним можно построить более сложный протокол.

HTTP/1.1 и HTTP/2 обычно работают поверх TCP, HTTPS добавляет TLS. HTTP/3 использует QUIC поверх UDP; надёжность, потоки и защищённый обмен обеспечиваются QUIC. Поэтому утверждение «весь веб работает только через TCP» слишком грубое.

Адрес 127.0.0.1 означает локальный компьютер. Привязка сервера к 0.0.0.0 означает прослушивание всех IPv4-интерфейсов, а не специальный адрес сайта. Доступ извне дополнительно зависит от маршрутизации, firewall и NAT. Приватный адрес не равен публично доступному адресу.

DNS

DNS сопоставляет имена с записями. А содержит IPv4-адрес, AAAA - IPv6, CNAME - псевдоним имени, MX - почтовый маршрут, TXT - текстовые данные для различных проверок. Рекурсивный resolver ищет ответ через DNS-иерархию и кеширует его. TTL ограничивает время кеширования, поэтому изменение записи может быть видно не всем одновременно.

```
dig example.com A
dig example.com AAAA
curl -I https://example.com
```

Примеры используют публичный демонстрационный домен. На Linux команда `ss -ltn` показывает слушающие TCP-сокеты; на macOS для похожей задачи часто используют `lsof -iTCP -sTCP:LISTEN`. Наличие программы в системе нужно проверить отдельно.

Поиск неисправности по слоям

Если домен не разрешается, начни с DNS. Connection refused обычно означает, что по выбранному адресу и порту соединение отклонено. Таймаут может быть связан с фильтрацией, маршрутом или перегрузкой. HTTP 500 означает, что до HTTP-сервера запрос уже дошёл. Успех ping не гарантирует доступность HTTPS, а отсутствие ответа ping не доказывает, что сайт недоступен: ICMP может фильтроваться.

Типичная ошибка. Искать проблему в React, когда backend слушает только локальный интерфейс другой машины.

Практика. Для трёх симптомов - ошибка DNS, отказ соединения, HTTP 404 - укажи первый проверяемый слой.

Самопроверка. Почему открытый порт ещё не означает исправную бизнес-функцию?

ГЛАВА 05

05. TLS, HTTPS и SSH

Что защищает TLS

TLS создаёт канал с конфиденциальностью, контролем целостности и проверкой подлинности участников в рамках выбранного режима. В обычном HTTPS браузер проверяет сервер. Сертификат связывает имя с открытым ключом через цепочку доверия; клиент проверяет имя, сроки и доверенную цепочку. Закрытый ключ сервера не отправляют посетителю. См. источник S2.

Упрощённо handshake согласует параметры, позволяет проверить сторону и получить общие ключи. Затем данные защищаются симметричным шифрованием. В современном TLS обмен ключами и подтверждение подлинности - разные части процесса; объяснение «весь трафик шифруется открытым ключом сертификата» неверно.

HTTPS защищает передачу между точками завершения TLS. Если TLS завершается на reverse proxy, дальнейшее соединение до backend нужно рассматривать отдельно. HTTPS не исправляет XSS, неверные права доступа и хранение паролей открытым текстом. HSTS сообщает браузеру требование использовать HTTPS для хоста; его включают, когда HTTPS уже устойчиво работает.

SSH: пользователь и удалённая машина

SSH даёт защищённый доступ к удалённой оболочке, передачу файлов и туннелирование. Есть две разные проверки: клиент убеждается, что подключился к нужному серверу, а сервер проверяет пользователя. Отпечаток host key следует сверять через доверенный канал; изменение ключа может быть законным после переустановки, но требует выяснения причины. См. S3.

```
ssh-keygen -t ed25519 -f ~/.ssh/study_ed25519
ssh -i ~/.ssh/study_ed25519 deploy@server.example
sftp -i ~/.ssh/study_ed25519 deploy@server.example
```

Это шаблон для твоего сервера: server.example нужно заменить. Не перезаписывай существующий ключ. Файл без .pub - закрытый ключ, .pub - открытый; на сервер помещают открытый ключ в authorized_keys соответствующего пользователя. Закрытый ключ защищают правами доступа и, как правило, парольной фразой.

Туннель на примере базы

```
ssh -N -L 127.0.0.1:15432:127.0.0.1:5432 \
  deploy@server.example
```

Локальный порт 15432 пересылается через SSH к порту 5432 с точки зрения удалённой машины. Это позволяет работать с доступной ей БД без публичного открытия её порта. Права и пароль самой БД всё равно нужны.

Типичная ошибка. Отключать проверку сертификата или host key, чтобы предупреждение исчезло, не выяснив причину.

Практика. Нарисуй, где лежат открытый и закрытый ключи пользователя и где клиент хранит доверие к серверу.

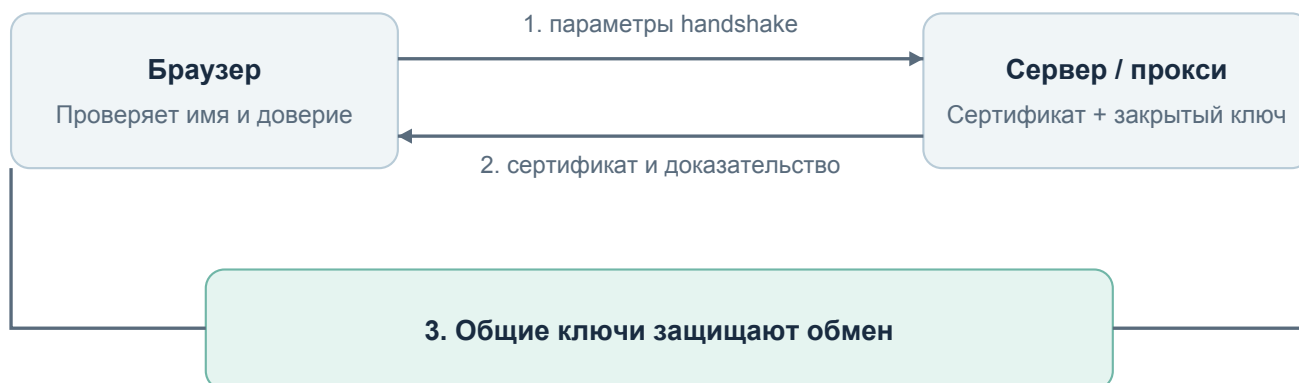
Самопроверка. Почему TLS и SSH нельзя считать взаимозаменяемыми названиями одного протокола?

СХЕМА 02 / К ГЛАВЕ 05

TLS и SSH: что именно проверяется

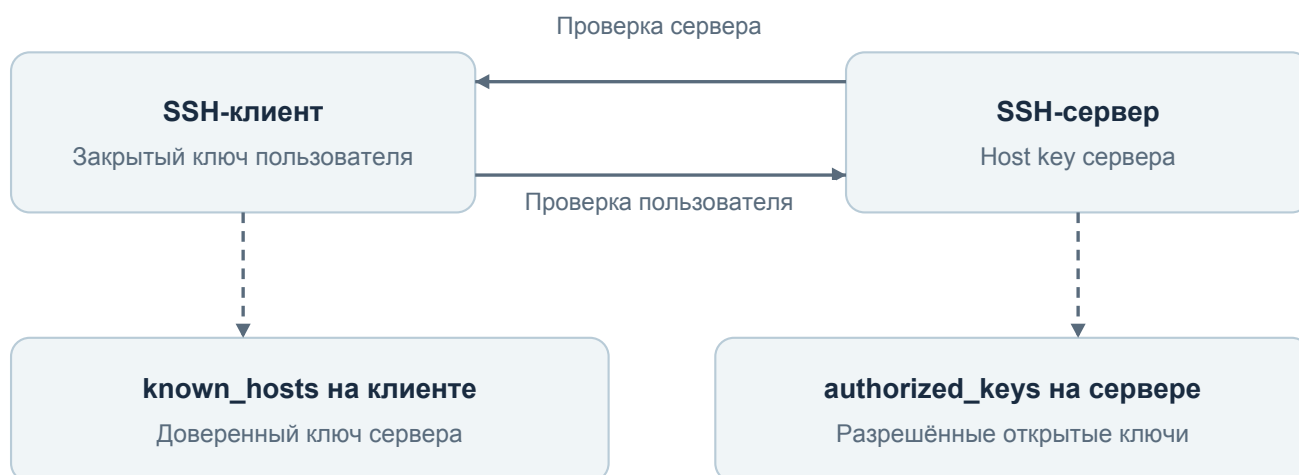
На одной странице две схемы: TLS для защищённого обмена и SSH для доступа к серверу. Handshake показан концептуально, без всех сообщений протокола.

А. HTTPS: канал для обмена с приложением



Упрощено: согласование параметров и ключей, проверка сервера, затем защищённые данные. Закрытый ключ не передаётся.

Б. SSH: две независимые проверки



Как читать схему

В TLS проверяется подлинность сервера и устанавливаются ключи канала. В SSH клиент проверяет host key, а сервер - пользователя. Открытый ключ пользователя не является ключом сервера.

Проверь себя. Где находится закрытый ключ пользователя и почему его нельзя копировать в `authorized_keys`?

ГЛАВА 06

06. HTTP, cookies и кеш браузера

Запрос как сообщение

HTTP-запрос содержит метод, адрес ресурса, заголовки и иногда тело. Ответ содержит статус, заголовки и иногда тело. GET читает представление ресурса; POST часто создаёт ресурс или запускает действие; PUT задаёт состояние ресурса по известному адресу; PATCH описывает частичное изменение; DELETE удаляет связь с ресурсом согласно контракту API. Семантика важнее названия функции в коде.

```
POST /orders HTTP/1.1
Host: shop.example
Content-Type: application/json
Accept: application/json

{"items": [{"productId": "p1", "quantity": 2}]}
```

Это текстовая иллюстрация HTTP/1.1, не полная инструкция ручной передачи по сокету. HTTP/2 использует двоичные кадры и мультиплексирование; HTTP/3 переносит обмен на QUIC. Приложение продолжает работать с методами, заголовками и статусами. Базовая модель запрос-ответ описана в S1.

Как выбирать статус

200 - успешный ответ, 201 - создание, 204 - успех без тела, 304 - можно использовать ранее сохранённое представление после условного запроса. 400 подходит для некорректного запроса, 401 - отсутствующей или непригодной аутентификации, 403 - отказа в доступе, 404 - отсутствующего ресурса. 409 описывает конфликт с текущим состоянием, 422 часто используют для семантически неприемлемых данных, 429 - превышения лимита. 500 - внутренняя ошибка; 502, 503 и 504 часто встречаются при проблемах шлюза, доступности и ожидания upstream.

Cookies и состояние

HTTP сам по себе не связывает бизнес-состояние последовательных запросов. Cookie позволяет браузеру сохранять и отправлять небольшое значение по правилам домена, пути и безопасности. Часто это непрозрачный идентификатор серверной сессии. HttpOnly ограничивает чтение из JavaScript, Secure требует защищённую передачу, SameSite регулирует отправку в межсайтовых сценариях. Ни один атрибут сам по себе не заменяет всю защиту приложения.

Условные запросы

Cache-Control управляет кешированием. no-store запрещает хранение; no-cache допускает хранение, но требует проверки перед повторным использованием. ETag обозначает версию представления; If-None-Match позволяет получить 304 вместо повторной передачи тела. Персональные ответы не должны случайно попасть в общий публичный кеш.

Типичная ошибка. Всегда возвращать 200 и прятать любую ошибку внутри JSON: посредникам и клиентам труднее различать успех и сбой.

Практика. Спроектируй ответы на создание заказа, отсутствие заказа, чужой заказ и повторную отмену.

Самопроверка. Чем отличаются no-cache и no-store?

ГЛАВА 07

07. HTML, CSS и доступность

Семантика до оформления

HTML описывает смысл документа. Заголовки создают иерархию, nav обозначает навигацию, main - основное содержимое, button - действие, a - переход. Правильный элемент уже имеет ожидаемое поведение клавиатуры и вспомогательных технологий. Замена кнопки на div требует самостоятельно восстанавливать это поведение.

Форма объединяет поля и отправку. Label должен быть связан с полем; сообщения об ошибках должны объяснять исправление и быть доступны программе чтения экрана. Placeholder не заменяет постоянную подпись. Проверка браузера улучшает опыт, но не отменяет серверную валидацию.

```
<form>
  <label for="quantity">Количество</label>
  <input id="quantity" name="quantity" type="number"
    min="1" max="20" value="1" required>
  <button type="submit">Добавить в корзину</button>
</form>
```

Как CSS вычисляет результат

Каскад определяет, какие объявления применяются с учётом происхождения, важности, слоёв, специфичности и порядка. Наследование передаёт некоторые свойства потомкам. Блочная модель включает content, padding, border и margin. При box-sizing: border-box заданная ширина включает внутренние отступы и границу.

Flexbox удобен для распределения элементов вдоль основной оси, Grid - для сетки строк и колонок. Адаптивность означает, что контент остаётся доступен при разных размерах, увеличении масштаба и длинном тексте; это больше, чем один media query для телефона.

```
* { box-sizing: border-box; }
.catalog {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(min(100%, 16rem), 1fr));
  gap: 1rem;
}
button:focus-visible {
  outline: 3px solid #126b82;
  outline-offset: 3px;
}
```

Доступность как часть корректности

Проверь весь сценарий клавиатурой, видимость фокуса, подписи полей, контраст и осмысленные альтернативы изображениям. После открытия диалога фокус должен попасть внутрь, после закрытия - вернуться логично. Ошибка не должна обозначаться одним цветом. Анимацию учитывают с учётом prefers-reduced-motion; смысл интерфейса не должен зависеть от движения.

Типичная ошибка. Фиксированная высота карточки обрезает длинное название или увеличенный текст.

Практика. Сверстай карточку товара и форму заказа без JavaScript; пройди их только Tab, Shift+Tab и Enter.

Самопроверка. Когда нужен link, а когда button?

ГЛАВА 08

08. JavaScript, DOM и асинхронность

Язык и окружение

JavaScript предоставляет значения, функции, объекты, массивы, замыкания и модули. DOM и fetch предоставляет окружение браузера, а не сам синтаксис языка. Замыкание сохраняет доступ к лексическому окружению функции; это полезно для обработчиков, но может удерживать данные в памяти дольше ожидаемого.

Различай null как явное отсутствие значения и undefined как отсутствие присвоенного значения в типичных сценариях. Используй строгое сравнение осмысленно. const запрещает переназначить переменную, но не делает объект неизменяемым. Методы map, filter и reduce выражают преобразования; побочные эффекты внутри них затрудняют чтение.

Promise и await

Promise представляет результат, который появится позже: выполнение или отклонение. await приостанавливает текущую асинхронную функцию до результата, но не блокирует весь сервер или браузер. Сама тяжёлая синхронная работа от наличия асинхронности быстрее не становится. Независимые операции можно выполнять конкурентно; зависимые должны ждать предыдущий результат.

```
async function loadProduct(id, signal) {
  const response = await fetch(
    `/api/products/${encodeURIComponent(id)}`,
    { signal }
  );
  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }
  return response.json();
}
```

fetch обычно не отклоняет Promise только из-за HTTP 404 или 500: нужно проверить статус. Ошибки сети и отмена обрабатываются отдельно. response.json() тоже может завершиться ошибкой, если тело не является корректным JSON.

События и гонки интерфейса

Пользователь быстро вводит два поисковых запроса. Ответ на первый может прийти позже второго и заменить свежие результаты устаревшими. Используй AbortController и/или проверку актуального идентификатора запроса. Отмена ожидания в браузере не гарантирует отмену уже выполненного изменения на сервере.

DOM обновляют через безопасные свойства, например.textContent для текста. Вставка пользовательской строки через innerHTML может превратить данные в исполняемую разметку. Делегирование событий позволяет одному обработчику обслуживать группу дочерних элементов; учитывая всплытие и event.target.

Типичная ошибка. Вызвать асинхронную функцию внутри forEach и ожидать, что внешний код дождётся всех результатов. Для ожидания используй подходящий цикл или Promise.all с ограничением конкурентности при большом объёме.

Практика. Сделай поиск с состояниями «загрузка», «пусто», «ошибка» и защитой от устаревшего ответа.

Самопроверка. Почему асинхронность не делает длительный вычислительный цикл неблокирующим?

ГЛАВА 09

09. TypeScript и границы доверия

Что дают типы

TypeScript проверяет согласованность кода до выполнения. Интерфейсы и type описывают форму данных, union - допустимые варианты, generics - связь типов между входом и выходом. Например, функция `identity<T>(value: T): T` сохраняет тип переданного значения. Типы помогают выражать ограничения и обнаруживать невозможные операции раньше.

После компиляции типы в основном исчезают. Утверждение «это Order» через `as` не проверяет ответ сервера. Внешние данные начинаются как `unknown` и проходят runtime-валидацию. `any` отключает существенную часть проверки; его распространение делает типизированный проект похожим на нетипизированный. Основы языка и его границы см. в S4.

```
type Result<T> =
  | { ok: true; value: T }
  | { ok: false; code: 'INVALID_QUANTITY' };

function parseQuantity(raw: unknown): Result<number> {
  if (typeof raw !== 'number' ||
      !Number.isInteger(raw) || raw < 1 || raw > 20) {
    return { ok: false, code: 'INVALID_QUANTITY' };
  }
  return { ok: true, value: raw };
}
```

Ветвление по `ok` сужает тип. Если результат успешный, `value` существует; иначе доступен `code`. Такая модель полезна для ожидаемых отказов, которые вызывающий код обязан разобрать.

Моделирование состояния

Вместо независимых флагов `isPaid`, `isCancelled` и `isPending` можно использовать явные варианты статуса. Иначе легко получить противоречивое состояние: заказ одновременно отменён и оплачен. Если некоторые поля существуют только в конкретном состоянии, включи их в соответствующую ветку `union`.

```
type OrderState =
  | { status: 'pending' }
  | { status: 'paid'; paymentId: string }
  | { status: 'cancelled'; reason: string };
```

Это помогает коду, но не заменяет ограничения и правила базы. Другой процесс может записать данные мимо TypeScript. Настройка `strict` усиливает проверки; `nullable`-значения нужно обрабатывать явно. Библиотеки схем, например `Zod`, помогают связать runtime-проверку с типом, но бизнес-проверка «товар ещё доступен» всё равно требует актуального состояния.

Типичная ошибка. Считать скомпилировавшийся код доказательством отсутствия ошибок данных или логики.

Практика. Опиши состояние загрузки каталога `union`-типом так, чтобы ошибка и успешные данные не появлялись одновременно без явного решения.

Самопроверка. Почему JSON от собственного backend тоже следует проверять на границе?

ГЛАВА 10

10. Компоненты, состояние и архитектура frontend

Компонент как единица интерфейса

Компонент соединяет представление с поведением и имеет явный вход. Props передают данные, события сообщают о намерении пользователя, состояние хранит изменяющуюся информацию. Один компонент не должен одновременно знать детали верстки кнопки, SQL-схему и правила платежного провайдера. React рассматривает UI как композицию компонентов; его вводный материал приведён в S5.

Локальное состояние относится к одному месту: открыт ли диалог. Общее состояние нужно нескольким частям: выбранная организация. Серверное состояние имеет внешний источник истины: каталог и заказы; для него важны кеш, устаревание, повторная загрузка и ошибки. Не копируй вычисляемое значение без необходимости: сумма корзины выводится из строк и цен.

```
import { useState } from 'react';

export function Quantity() {
  const [value, setValue] = useState(1);
  return (
    <button onClick={() => setValue(v => v + 1)}>
      Количество: {value}
    </button>
  );
}
```

Функциональное обновление использует предыдущее состояние. В реальном поле количества добавь ограничения и возможность уменьшения. Эффекты нужны для синхронизации с внешними системами, например подпиской, а не для каждого вычисления из props.

Организация проекта

Группируй код вокруг устойчивых функций: catalog, cart, orders, account. Общие элементы интерфейса выноси, когда они действительно имеют общий контракт. Работа с API должна иметь понятную границу, чтобы компоненты не повторяли сборку заголовков и разбор ошибок. Маршрут отвечает за навигационный контекст, но не обязан содержать всю бизнес-логику страницы.

Состояния и восстановление

Для каждой загрузки продумай pending, success, empty и error. При optimistic update интерфейс показывает ожидаемый результат раньше ответа; при неудаче нужно откатить изменение или явно показать конфликт. Для оплаты такая оптимистичность не означает право объявить заказ оплаченным без подтверждения.

Error boundary в React изолирует некоторые ошибки рендеринга потомков, но не является универсальным перехватчиком всех ошибок обработчиков и асинхронных операций. Для запросов и событий нужна своя обработка.

Типичная ошибка. Хранить каждое поле формы в глобальном store: это расширяет связи без реальной пользы.

Практика. Раздели страницу заказа на компоненты и подпиши источник истины каждого значения.

Самопроверка. Чем состояние «данные отсутствуют» отличается от «данные ещё не загрузились»?

СХЕМА 03 / К ГЛАВЕ 10

Где хранить состояние интерфейса

Разделение источников данных помогает избежать противоречий и лишних связей между компонентами.



Как читать схему

Открытый диалог обычно относится к локальному UI. Данные заказа принадлежат серверу; их копии требуют правил загрузки и обновления. Итог корзины можно вычислить из строк.

Проверь себя. Нужно ли отдельно хранить сумму корзины, если её можно вычислить из товаров и количества?

ГЛАВА 11

11. CSR, SSR, SSG, сборка и производительность UI

Где появляется HTML

При CSR основную работу построения интерфейса выполняет JavaScript в браузере. При SSR сервер формирует HTML для запроса, а браузер может затем подключить интерактивность. SSG создаёт HTML заранее при сборке; некоторые системы позволяют обновлять отдельные страницы позже. Подходы можно сочетать для разных частей приложения.

Гидратация связывает клиентскую логику с уже полученным HTML. Если первый клиентский результат отличается от серверного, возникают несоответствия. Например, прямой вывод текущего времени или случайного числа на обеих сторонах может дать разную разметку. SSR требует учитывать, что `window` и `document` на сервере отсутствуют.

Выбор на примере магазина

Описание редко меняющегося товара можно подготовить заранее. Кабинет пользователя требует персональных данных и строгого разделения кешей. Интерактивный редактор может выполнять большую часть работы на клиенте. Выбор зависит от требований к первому показу, индексации, интерактивности, стоимости сервера и актуальности данных.

Каталог:

HTML с описанием -> быстрый первый показ
Клиентский запрос -> актуальный остаток

Личный кабинет:

Проверка сессии -> персональные данные
Общий CDN-кеш для ответа не используется

Сборка приложения

Менеджер пакетов устанавливает зависимости, транспилятор преобразует синтаксис, `bundler` связывает модули и готовит ресурсы. `Tree shaking` может убрать неиспользуемые части при подходящих условиях. `Code splitting` делит код, `lazy loading` откладывает загрузку нужной части. `Source maps` помогают связать ошибки собранного кода с исходниками; доступ к ним оценивают с учётом содержимого.

Измеряй сетевой вес, время выполнения JS, задержку реакции и стабильность раскладки. Задай размеры изображений, чтобы интерфейс не прыгал при загрузке. Сжатие картинок часто полезнее микрооптимизации компонента. Кеширование статических файлов с хешем в имени позволяет выпускать новые версии без смещения ресурсов.

Типичная ошибка. Считать SSR автоматическим ускорением любого приложения: дорогой серверный рендеринг и большой клиентский `bundle` могут ухудшить результат.

Практика. Для каталога, кабинета и редактора товара выбери способ рендеринга и запиши по два аргумента.

Самопроверка. Почему серверный HTML ещё не означает, что все кнопки уже интерактивны?

ГЛАВА 12

12. Фреймворки: карта и критерии выбора

Фреймворк, библиотека и runtime

Библиотеку вызывает твой код; фреймворк обычно задаёт жизненный цикл и вызывает написанные тобой обработчики. Runtime исполняет программу: Node.js - среда выполнения JavaScript, а не веб-фреймворк. ORM работает с хранением данных; reverse проху принимает и пересылает HTTP. Эти инструменты решают разные задачи и не заменяют друг друга.

Выбор фреймворка - выбор набора соглашений и возможностей. Минималистичный инструмент даёт свободу, но многие решения нужно согласовать самостоятельно. Более комплексный предоставляет структуру, интеграции и ограничения. Сравнивай опыт команды, тип продукта, экосистему, модель развёртывания и стоимость сопровождения; не выбирай по одному синтетическому benchmark.

Frontend и full-stack

React - библиотека компонентов; маршрутизация и получение данных зависят от окружения. **Vue** объединяет реактивность и компонентную модель. **Angular** предлагает комплексную структуру приложения и встроенные механизмы. **Svelte** использует компиляцию компонентов. Общие знания - события, состояние, жизненный цикл, композиция и доступность - переносятся между ними.

Next.js, **Nuxt** и **SvelteKit** добавляют к соответствующим экосистемам маршруты, серверный рендеринг и серверные возможности. Возможность написать серверный обработчик рядом с интерфейсом не отменяет проверки прав, транзакций и границ модулей. Конкретные правила кеширования и выполнения зависят от версии и режима развёртывания.

Backend по языкам

JavaScript/TypeScript: Express предоставляет небольшой набор HTTP-механизмов; Fastify делает акцент на плагинах и схемах; NestJS задаёт модульную организацию, controllers и dependency injection. Это разные уровни структуры, а не шкала «плохой - хороший».

Python: Django объединяет ORM, маршрутизацию, административный интерфейс и другие веб-механизмы; FastAPI ориентирован на API и использует аннотации типов для описания и проверки моделей; Flask даёт компактную основу с расширениями. Обзор Django и FastAPI см. S7 и S8.

Java/Kotlin: Spring Boot упрощает настройку приложений экосистемы Spring. **C#:** ASP.NET Core предоставляет HTTP pipeline, DI и средства построения API. **PHP:** Laravel и Symfony предлагают структуру веб-приложений. **Ruby:** Rails опирается на соглашения и интегрированный набор инструментов. **Go:** Gin и Echo дополняют возможности стандартного net/http. **Rust:** Axum и Actix Web помогают строить HTTP-сервисы с моделью типов и конкурентности Rust.

Фреймворк в работе: небольшой пример

Чтобы увидеть роль фреймворка, рассмотрим маленький обработчик на FastAPI. Это отдельный фрагмент Python, не часть TypeScript-проекта. Для запуска нужны совместимые версии Python, FastAPI, Pydantic и ASGI-сервер; команду запуска выбирают по установленной версии. Данные пока не сохраняются: endpoint демонстрирует маршрутизацию, модель входа и формирование ответа.

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class Item(BaseModel):
    product_id: str = Field(min_length=1)
    quantity: int = Field(ge=1, le=20, strict=True)

@app.post("/items", status_code=201)
def create_item(item: Item):
    return {"productId": item.product_id,
            "quantity": item.quantity}
```

Декоратор связывает путь и метод с функцией. Модель проверяет форму входа и диапазон количества, а return задаёт данные ответа. При этом код ещё не проверяет существование товара, не знает пользователя и не защищает остатки от гонки. Эти обязанности не исчезают из-за автоматической валидации.

В NestJS похожий сценарий организуют через controller, DTO и service, а проверку входа подключают подходящим механизмом валидации. В Django можно использовать модель данных, view и слой сценария; для полноценного REST API часто добавляют соответствующий инструмент экосистемы. Смысл остаётся тем же: преобразовать протокол, проверить границу и выполнить правило.

Какие механизмы изучать в выбранном инструменте

Начни с маршрутов и жизненного цикла запроса. Затем изучи настройку приложения, зависимости, валидацию, обработку ошибок и тестовый клиент. После этого подключи БД, миграции, аутентификацию и graceful shutdown. Так ты осваиваешь систему целиком, а не запоминаешь только синтаксис контроллера.

Для данных встречаются Prisma, Drizzle, SQLAlchemy, Hibernate, Entity Framework Core; для валидации - Zod, Pydantic и Jakarta Bean Validation; для фоновых задач - BullMQ, Celery и Sidekiq. Понятия важнее конкретных API: связь сущностей, миграция, проверка схемы, подтверждение задания, повтор.

Пример выбора. Если команда знает Python и нужен кабинет с большим числом административных операций, интегрированная экосистема Django может снизить объём сборки инфраструктуры. Если цель - изучать один язык на клиенте и сервере, TypeScript позволяет сосредоточиться на общих принципах. Это сценарии рассуждения, а не универсальные рекомендации.

Типичная ошибка. Изучать пять backend-фреймворков поверхностно и не уметь обеспечить целостность одной операции.

Практика. Сравни два инструмента по пяти критериям твоего проекта и реализуй в одном из них небольшой CRUD.

Самопроверка. Что останется твоей ответственностью, даже если фреймворк автоматически создаёт API?

ГЛАВА 13

13. Внутри сервера: процессы, память и event loop

Процесс и поток

Операционная система запускает процесс с собственным адресным пространством и ресурсами. Потоки внутри процесса разделяют память; это облегчает обмен, но создаёт риски гонок. Конкурентность означает перекрывающееся продвижение задач, параллелизм - их фактическое одновременное исполнение. Один event loop может обслуживать много ожидающих операций без параллельного выполнения их JavaScript-кода.

Сервер создаёт слушающий сокет. При поступлении соединений ОС и runtime доставляют данные приложению. Фреймворк разбирает HTTP и вызывает обработчик. Пока приложение ждёт сеть или БД, неблокирующая модель позволяет заняться другим запросом. При блокирующей модели ожидание занимает поток; масштабирование зависит от количества потоков и стоимости каждого.

CPU-bound и I/O-bound

CPU-bound задача ограничена вычислениями: сложное преобразование изображения, архивирование, большой расчёт. I/O-bound - ожиданием внешней системы: запрос к базе или API. В Node.js event loop организует выполнение callback и обработку событий, а часть операций использует другие механизмы runtime и ОС. Нельзя свести всю среду к фразе «в ней всегда только один поток». См. S6.

```
// Плохо внутри обработчика запроса:
async function expensive() {
  let sum = 0;
  for (let i = 0; i < 1_000_000_000; i++) sum += i;
  return sum;
}
```

async здесь не отдаёт вычисление worker-потoku. Такая работа может надолго занять event loop. В зависимости от задачи нужны worker threads, отдельные процессы, очередь или более эффективный алгоритм.

Память и ресурсы

Стек хранит контекст вызовов; куча - динамические объекты в модели runtime. Сборщик мусора освобождает недостижимые объекты, но не исправляет бесконечно растущий Map, на который сохраняется ссылка. Файлы, сокеты, таймеры и подписки тоже требуют управления жизненным циклом.

Пул соединений повторно использует ограниченное число подключений к БД. Если каждый из 20 экземпляров сервера откроет по 50 соединений, общий спрос составит 1000. Увеличение числа приложений без учёта общей базы может усугубить отказ. Ожидание свободного соединения должно иметь ограничение.

Типичная ошибка. Лечить высокую задержку увеличением timeout, хотя причина - блокировка event loop или исчерпанный пул.

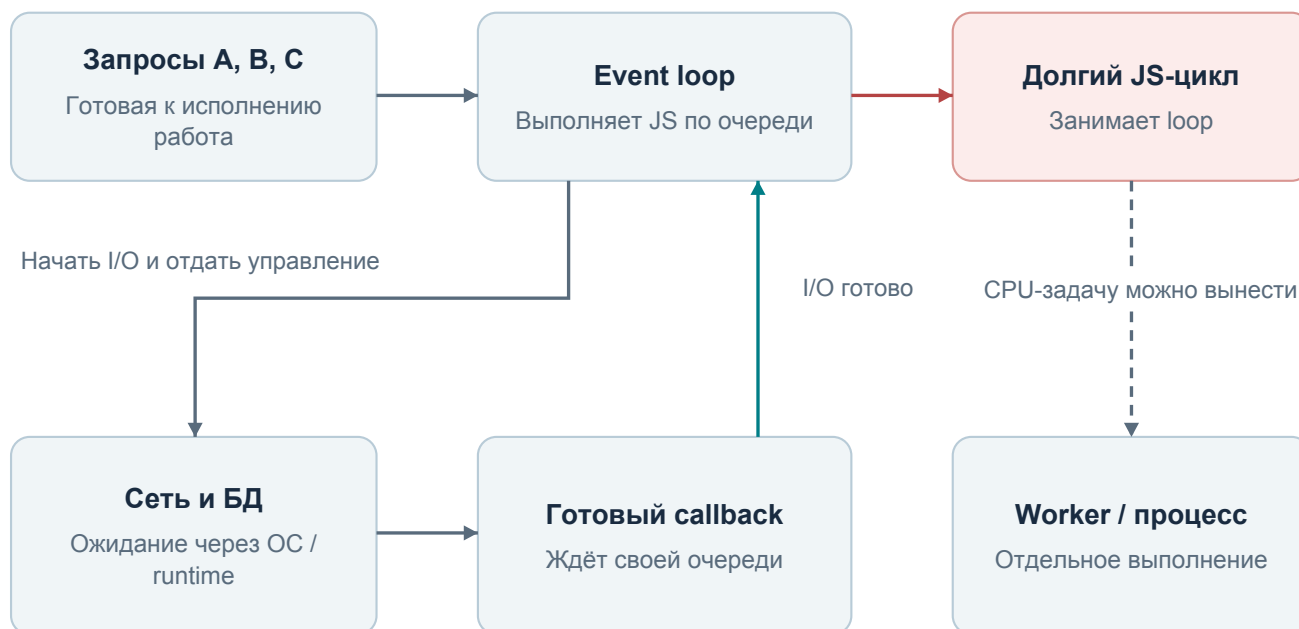
Практика. Раздели загрузку изображения, его перекодирование и запись результата по типам нагрузки.

Самопроверка. Почему сервер с небольшим потреблением CPU всё равно может отвечать медленно?

СХЕМА 04 / К ГЛАВЕ 13

Event loop: ожидание и вычисления

Схема относится к типичной модели Node.js. Детали runtime различаются; здесь важна разница между ожиданием I/O и выполнением JavaScript.

**async не создаёт новый поток**

await уступает выполнение при ожидании. Синхронный цикл без уступки продолжает занимать поток.

Как читать схему

Пока I/O ожидает завершения, loop может выполнять другую готовую работу. Длинный синхронный цикл занимает поток и задерживает остальные callback. Вынос CPU-задачи требует отдельного механизма.

Проверь себя. Почему добавление `async` перед функцией не устраняет блокирующий цикл?

ГЛАВА 14

14. Жизненный цикл запроса и middleware

От байтов к сценарию

После разбора HTTP запрос проходит цепочку инфраструктурных шагов. Обычно это идентификатор запроса, ограничения, аутентификация, маршрутизация, валидация и обработчик. Порядок влияет на поведение: проверка подписи webhook иногда требует исходного тела до преобразования JSON, а логирование должно видеть и ошибки ранних этапов.

Middleware - звено цепочки, которое читает или изменяет контекст, продолжает обработку либо завершает запрос. Ему подходят сквозные задачи: журналирование, CORS, сессии, лимиты. Сложное правило расчёта скидки лучше вынести в отдельную бизнес-функцию, чтобы оно не было привязано к механике HTTP.

Запрос

- > request ID и таймер
- > лимит тела и разбор
- > проверка сессии
- > валидация DTO
- > controller
- > application service
- > repository / внешний адаптер
- > сериализация ответа
- > запись результата в лог

Controller, service, repository

Controller переводит протокол в вызов сценария: извлекает вход и пользователя, вызывает service, формирует ответ. Service координирует бизнес-действие и транзакцию. Repository предоставляет осмысленные операции хранения, скрывая SQL или ORM там, где это действительно помогает. Такое разделение - полезный вариант, а не обязательные три папки для любого проекта.

```
// Схема: интерфейсы framework и orders заданы проектом.
async function createOrderHandler(req, res) {
  const input = parseCreateOrder(req.body);
  const order = await orders.create({
    customerId: req.user.id,
    items: input.items
  });
  res.status(201).json({ id: order.id, status: order.status });
}
```

customerId берётся из проверенного контекста, а не из свободного поля JSON. При этом аутентификация сама по себе не отвечает на вопрос, имеет ли пользователь право на конкретное действие: эта проверка остаётся частью сценария.

Время жизни запроса

У запроса есть бюджет времени. Бесконечно ждать зависимость нельзя. При отмене нужно по возможности передать сигнал дальше, освободить ресурсы и корректно определить судьбу операции. После отправки заголовков нельзя произвольно заменить уже начатый ответ на другой статус. Центральный error handler формирует безопасный ответ и сохраняет диагностический контекст.

Типичная ошибка. Перехватывать исключение, писать лог и забывать завершить ответ: клиент ждёт до таймаута.

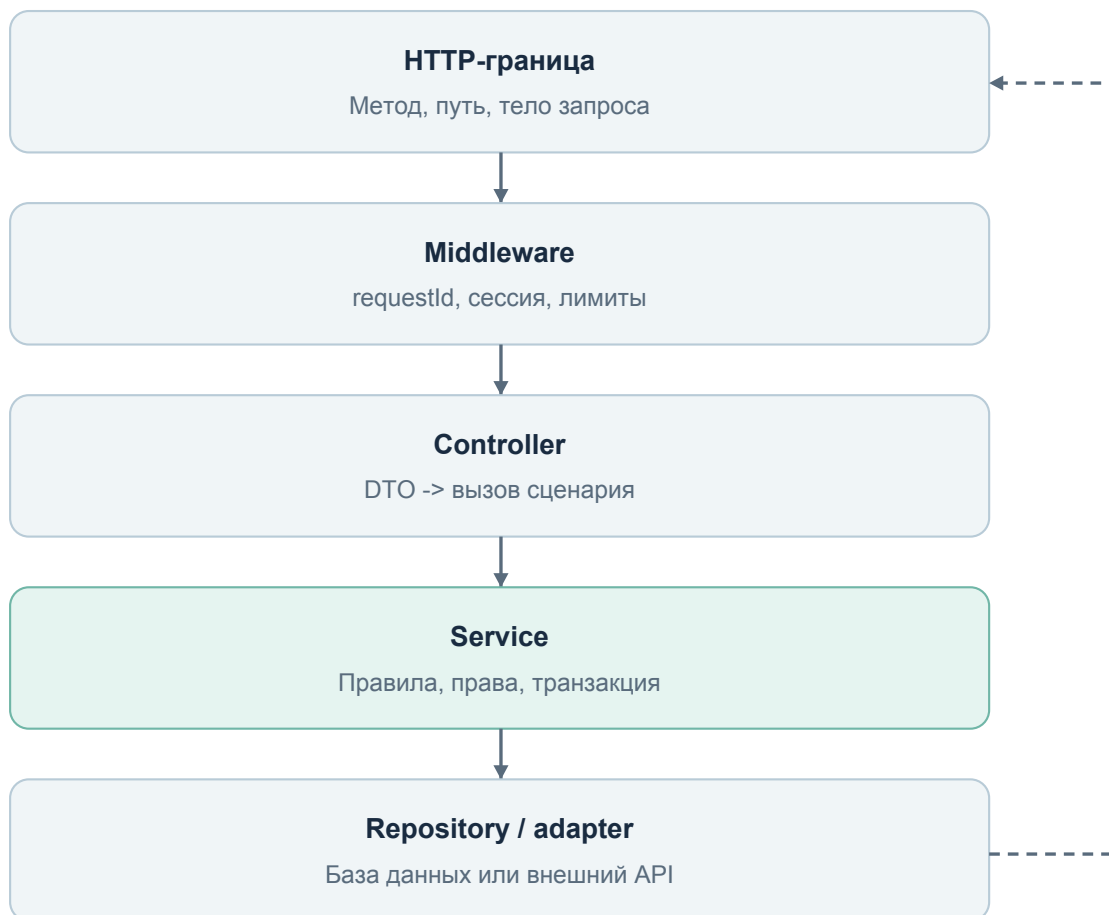
Практика. Подпиши ответственность каждого звена для запроса «отменить заказ».

Самопроверка. Почему проверка JWT в middleware не заменяет проверку владельца заказа?

СХЕМА 05 / К ГЛАВЕ 14

Кто за что отвечает на сервере

Каждый блок преобразует вход и передаёт работу следующему уровню. Это пример организации, а не обязательное число папок.



Ответ и ошибки возвращаются через границы. Общая обработка ошибок переводит внутреннюю причину в безопасный HTTP-ответ.

Как читать схему

Controller связывает HTTP со сценарием. Service координирует правила и транзакцию. Repository и адаптеры выполняют технические операции. Проверки доступа нельзя оставлять только в UI.

Проверь себя. В каком блоке следует считать скидку, а в каком - выбирать HTTP-статус?

ГЛАВА 15

15. Бизнес-логика и состояния заказа

Сценарий важнее CRUD

CRUD описывает технические операции с записями. Бизнес-сценарий объясняет намерение: оформить заказ, зарезервировать товар, подтвердить оплату, отменить доставку. Операция `updateOrder` с произвольным объектом может позволить комбинации, которые бизнес запрещает. Команды `cancelOrder` и `confirmPayment` задают более точный контракт.

Инвариант - условие, которое должно сохраняться после допустимой операции. Например, количество положительно, итог неотрицателен, оплаченный заказ содержит подтверждение платежа.

Предусловие проверяется перед действием, постусловие - после успешного завершения. Эти правила нужно выражать кодом и, где возможно, ограничениями базы.

Конечный автомат

Состояния и переходы делают правила видимыми. Для упрощённого магазина допустимы `pending` -> `paid`, `pending` -> `cancelled` и `paid` -> `shipped`. Возврат после оплаты - отдельный процесс, а не молчаливое стирание факта оплаты. Реальная модель может быть сложнее: частичные платежи и отгрузки требуют дополнительных сущностей.

```
type Status = 'pending' | 'paid' | 'cancelled' | 'shipped';

function cancel(status: Status): Status {
  if (status === 'cancelled') return status;
  if (status !== 'pending') {
    throw new Error('ORDER_CANNOT_BE_CANCELLED');
  }
  return 'cancelled';
}
```

Пример показывает правило в памяти. Для реальной конкуренции переход нужно подтвердить атомарно в БД: `UPDATE ... WHERE status = 'pending'`, а затем проверить число обновлённых строк. Между отдельным чтением и записью другой запрос мог уже подтвердить оплату.

Цена, время и границы системы

Сервер получает актуальные цены, вычисляет итог и сохраняет снимок цены строки заказа. Иначе изменение каталога задним числом изменит историю покупки. Деньги хранят как целые минимальные единицы либо подходящий `decimal` с валютой; двоичный `float` может дать ошибку округления. Правила округления и распределения скидки должны быть явными.

Момент события удобно хранить как `timestamp` с однозначной временной интерпретацией; локальный день рождения - другая сущность, которой не нужен часовой пояс. Для расписания «каждый день в 09:00» нужен часовой пояс пользователя и политика переходов времени, а не только UTC-смещение.

Типичная ошибка. Принимать итоговую стоимость из браузера или повторно вычислять старый заказ по сегодняшнему каталогу.

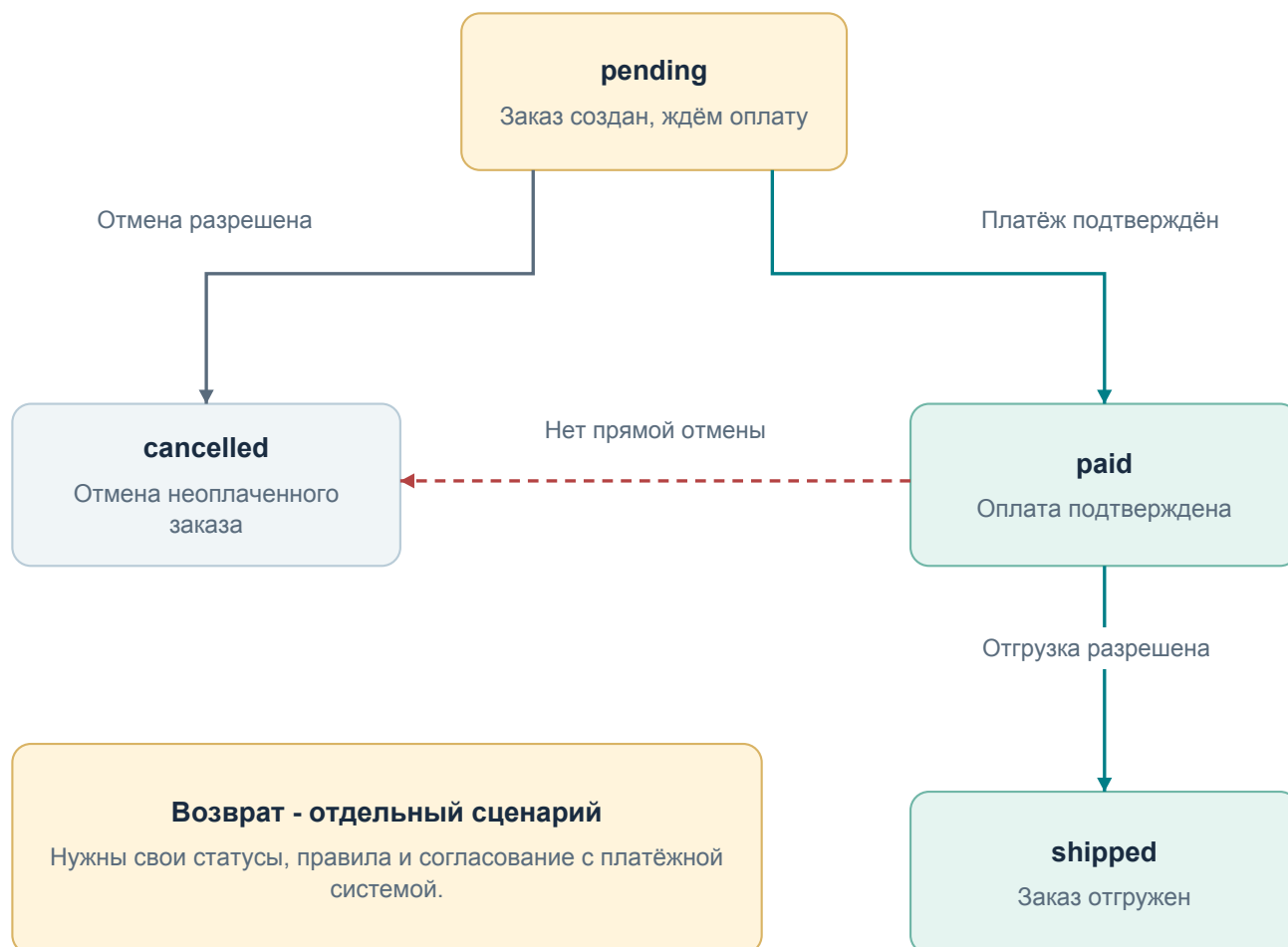
Практика. Нарисуй таблицу переходов заказа и опиши, кто может выполнить каждый переход.

Самопроверка. Почему нельзя всегда исправить ошибочный платёж простым откатом записи в своей базе?

СХЕМА 06 / К ГЛАВЕ 15

Жизненный цикл заказа

Конечный автомат делает допустимые изменения явными. Красная пунктирная стрелка обозначает запрещённый прямой переход.



Как читать схему

Из **pending** возможны оплата или отмена. После **paid** этот упрощённый сценарий допускает отгрузку. Возврат денег требует отдельного процесса. В БД переход подтверждается атомарно.

Проверь себя. Что делать, если оплата и отмена приходят одновременно?

ГЛАВА 16

16. Проектирование API и контрактов

API как обещание

Контракт описывает входы, выходы, ошибки и семантику повторов. DTO - форма данных на границе, которая может отличаться от внутренней модели. Не возвращай ORM-сущность целиком: в ней могут появиться поля, которые клиент не должен видеть. Явный serializer позволяет контролировать публичный формат.

REST строится вокруг ресурсов и семантики HTTP. RPC делает акцент на вызове операций. GraphQL даёт клиенту язык выбора данных через схему, но требует контроля стоимости запросов, доступа к полям и N+1. Ни один стиль автоматически не гарантирует хорошую архитектуру.

```
GET    /products?category=books&limit=20
POST   /orders
GET    /orders/{id}
POST   /orders/{id}/cancellation
```

Последний маршрут моделирует отмену как явное действие через создание связанного ресурса. Возможны и другие контракты; важно, чтобы ограничения и результат были понятны, а изменение статуса не обходило правила.

Валидация и пагинация

Сначала проверь структуру: типы, обязательные поля, длины, диапазоны, неизвестные поля по выбранной политике. Затем семантику: существует ли товар, разрешена ли валюта. Наконец проверь состояние, которое может меняться конкурентно, внутри подходящей атомарной операции.

Offset-пагинация проста, но глубокое смещение дорого и при изменениях записи могут дублироваться или пропускаться. Cursor-пагинация использует позицию в устойчивом порядке, например `created_at` и `id`. Порядок должен разрешать совпадающие значения; одного времени часто недостаточно. Лимит размера страницы защищает и сервер, и клиента.

Совместимость

Добавление необязательного поля часто совместимо, изменение смысла существующего - нет. Даже добавление `enum`-варианта может сломать клиента с исчерпывающим `switch`. Продумывай период миграции, наблюдай использование старых версий и документируй изменения. OpenAPI описывает HTTP-контракт и помогает генерации клиентов, но бизнес-смысл ошибок всё равно нужно объяснять.

Rate limiting ограничивает число операций по подходящему ключу: пользователю, токenu, IP или их сочетанию. Ограничение только по IP может несправедливо объединить пользователей за общим NAT. Для больших задач полезен 202 Accepted с ресурсом статуса; это означает принятие к обработке, а не успешное завершение.

Типичная ошибка. Менять тип поля `id` с числа на строку без оценки клиентов, кешей и документации.

Практика. Опиши контракт создания заказа: схему, статусы, три ошибки, повтор и доступ.

Самопроверка. Что должен узнать клиент из ответа 202, чтобы отслеживать операцию?

ГЛАВА 17

17. Идемпотентность и неопределённый результат

Повтор - нормальная ситуация

Клиент отправил создание заказа, сервер сохранил запись, но ответ потерялся. Для клиента это таймаут, для сервера - успех. Если повтор без защиты снова создаст заказ, возникнет дубликат. Таймаут сообщает об отсутствии результата вовремя, а не об отсутствии побочного эффекта.

Идемпотентная операция после повторов имеет тот же предполагаемый эффект, что после одного выполнения. Это не требует одинакового HTTP-ответа: повторный DELETE может вернуть другой статус, хотя целевое состояние остаётся «ресурс удалён». POST не получает идемпотентность автоматически; её можно добавить к контракту.

Ключ идемпотентности

Клиент генерирует ключ одной логической попытки и сохраняет его для повторов. Сервер связывает ключ с пользователем, типом операции, отпечатком входа и результатом. Уникальное ограничение защищает от двух одновременных обработчиков. Повтор того же ключа с другим телом должен быть отвергнут, иначе одна операция случайно подменит другую.

1. Получить (user_id, key, request_hash).
2. Атомарно занять ключ или прочитать существующий.
3. При другом hash вернуть конфликт.
4. Если завершено, вернуть сохранённый результат.
5. Если выполняется, применить договорённую политику ожидания.
6. Создание заказа и фиксацию результата согласовать в одной транзакции, если они в одной БД.

Схема требует проектирования срока хранения ключей и поведения после сбоя. Если ключ хранится меньше возможного интервала повторов, защита может исчезнуть слишком рано. Если после внешнего платежа процесс упал до фиксации локального результата, одной транзакции своей БД недостаточно: нужен ключ на стороне провайдера и/или сверка статуса.

Где ещё нужна идемпотентность

Webhook может прийти повторно; задание очереди может быть доставлено после падения worker; человек может дважды нажать кнопку. Блокировка кнопки улучшает интерфейс, но не заменяет серверную защиту. Дедупликация по event_id предотвращает повторную обработку конкретного события, а бизнес-инвариант защищает от разных событий с одинаковым эффектом.

Типичная ошибка. Создавать новый ключ при каждой сетевой попытке: сервер увидит разные операции.

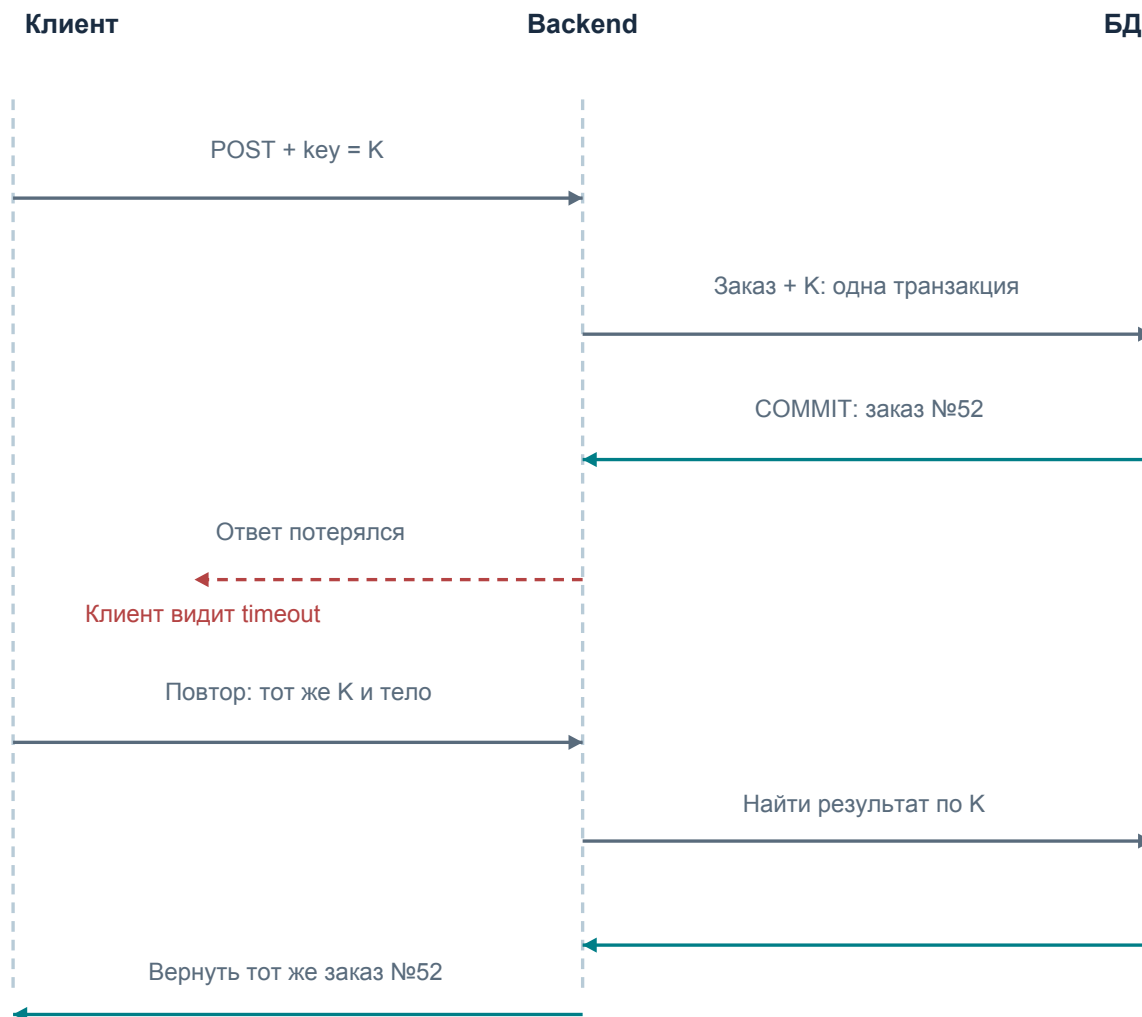
Практика. Опиши четыре сценария: первый запрос, тот же запрос, тот же ключ с другим телом, падение после внешней оплаты.

Самопроверка. Почему «повторить три раза любой неуспешный POST» опасно?

СХЕМА 07 / К ГЛАВЕ 17

Таймаут и безопасный повтор

Читай сверху вниз: вертикаль означает время, горизонтальные стрелки - сообщения и действия между участниками.



Как читать схему

Заказ уже зафиксирован, хотя клиент не получил ответ. Повтор с тем же ключом K и тем же входом возвращает сохранённый результат. Уникальное ограничение защищает от двух обработчиков одного ключа.

Проверь себя. Что случится, если клиент при повторе создаст новый ключ?

ГЛАВА 18

18. SQL, модель данных и ORM

Схема отражает смысл

Таблица представляет множество строк с определёнными полями и ограничениями. Primary key идентифицирует строку, foreign key связывает её с другой таблицей, UNIQUE ограничивает дубликаты, NOT NULL требует значение, CHECK задаёт условие. Ограничения работают независимо от того, какой клиент пишет в БД.

Связь один-ко-многим: один покупатель имеет много заказов. Многие-ко-многим обычно выражается промежуточной таблицей: заказ содержит товары через order_items. В строке заказа хранится количество и цена на момент покупки, потому что это свойства конкретного отношения, а не текущего товара.

```
CREATE TABLE products (  
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  title text NOT NULL,  
  price_minor bigint NOT NULL CHECK (price_minor >= 0),  
  stock integer NOT NULL CHECK (stock >= 0)  
);  
  
SELECT id, title, price_minor  
FROM products  
WHERE stock > 0  
ORDER BY id  
LIMIT 20;
```

Запросы и нормализация

SELECT читает данные, INSERT создаёт, UPDATE меняет, DELETE удаляет. JOIN соединяет связанные строки, GROUP BY группирует, HAVING фильтрует группы после агрегации. NULL обозначает отсутствие значения; для проверки используют IS NULL, а не равенство. Условия с NULL подчиняются трёхзначной логике, что особенно важно для фильтров.

Нормализация уменьшает противоречивое дублирование: адрес пользователя не нужно копировать в каждую произвольную запись. Но адрес доставки конкретного заказа часто должен быть историческим снимком. Денормализация добавляет дублирование ради чтения или истории; нужно заранее определить источник истины и способ обновления.

ORM и миграции

ORM отображает объекты и связи на операции БД. Она уменьшает повторяющийся код, но не отменяет SQL, индексы и транзакции. Удобный доступ order.items может незаметно породить дополнительные запросы. Миграция - версионизируемое изменение схемы; она нужна для воспроизводимого перехода между состояниями базы.

SQL и NoSQL выбирают по модели доступа и гарантиям. Документная база удобна для некоторых вложенных структур, key-value - для доступа по ключу. У NoSQL тоже есть схема, даже если она не полностью принуждается самой БД; просто часть правил переносится в приложение.

Типичная ошибка. Проверить уникальность email отдельным SELECT и считать это достаточной защитой при конкурентной регистрации.

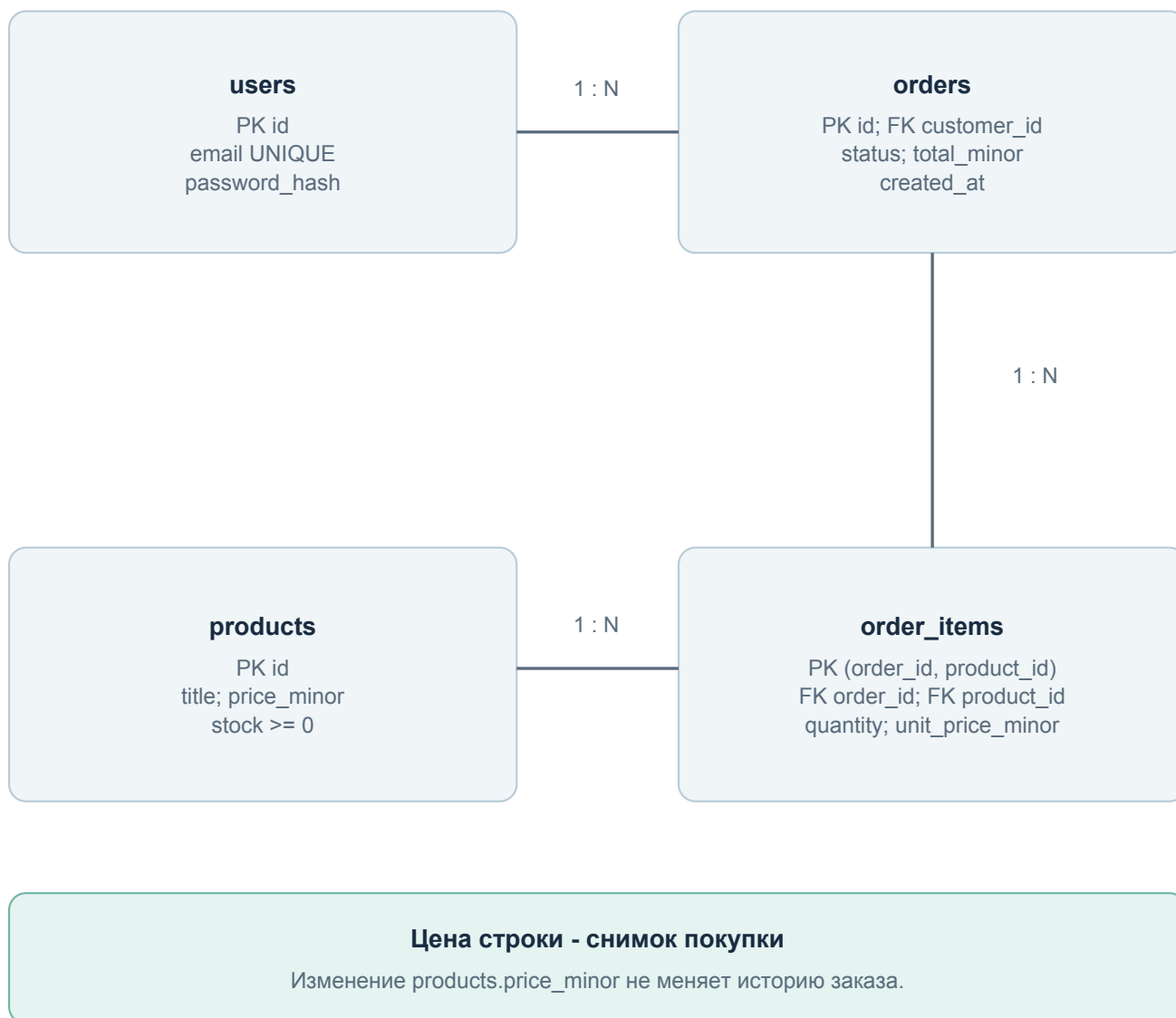
Практика. Спроектируй users, orders и order_items с ключами и ограничениями.

Самопроверка. Почему цена в order_items не является просто вредным дублированием products.price?

СХЕМА 08 / К ГЛАВЕ 18

Связи данных: пользователь, заказ, товар

PK - первичный ключ, FK - внешний ключ. Обозначение 1 : N означает связь одного объекта с несколькими записями.



Как читать схему

Заказ принадлежит пользователю и содержит строки. Каждая строка связывает заказ с товаром, но сохраняет собственную цену покупки. В этой учебной схеме один товар представлен одной строкой заказа.

Проверь себя. Почему нельзя всегда показывать цену старого заказа из текущей таблицы products?

ГЛАВА 19

19. Транзакции, изоляция и конкурентность

Что даёт транзакция

Транзакция объединяет несколько действий в единицу фиксации. Atomicity означает «всё или ничего» для её изменений в рамках БД. Consistency связана с сохранением заданных ограничений. Isolation определяет видимость конкурентных действий. Durability - сохранность зафиксированного результата в рамках настроенных гарантий хранения. ACID не превращает произвольную бизнес-логику в корректную автоматически.

Классическая гонка: два покупателя читают `stock = 1`, оба считают покупку допустимой и оба создают заказ. Одна предварительная проверка не защищает от изменения между чтением и записью.

```
BEGIN;
UPDATE products
SET stock = stock - 1
WHERE id = 42 AND stock >= 1
RETURNING id;
-- Если строк нет: приложение делает ROLLBACK.
-- Если строка есть: INSERT заказа и строки заказа.
COMMIT;
```

Пример показывает атомарное условное уменьшение остатка. При заказе нескольких товаров нужны единая транзакция, проверка каждого результата и согласованный порядок захвата ресурсов. Если любой товар недоступен, изменения откатывают.

Изоляция

При Read Committed отдельные запросы могут видеть разные зафиксированные состояния. Repeatable Read в PostgreSQL использует устойчивый снимок в рамках транзакции, но не делает любой набор бизнес-проверок сериализуемым. Serializable стремится дать результат, эквивалентный некоторому последовательному выполнению, и может потребовать повторить целую транзакцию после ошибки сериализации. Точные гарантии различаются между СУБД; см. S9.

Dirty read - чтение незавершённых изменений, non-repeatable read - изменение ранее прочитанной строки при повторе, phantom - изменение множества строк по условию. Lost update - потеря одного обновления. Write skew может нарушить правило, которое зависит от нескольких строк, даже если транзакции меняют разные строки.

Виды блокировок

Пессимистический подход заранее блокирует нужные записи, например `SELECT ... FOR UPDATE`. Оптимистический проверяет версию при записи: `UPDATE ... WHERE version = old_version`. Если никто не обновлён, данные изменились, и нужно разрешить конфликт. Deadlock - цикл взаимного ожидания ресурсов; БД прерывает одну из транзакций, после чего приложение решает вопрос повторения.

Типичная ошибка. Держать транзакцию и блокировки открытыми во время длительного вызова платёжного API.

Практика. В двух сессиях БД воспроизведи конкурентную покупку последнего товара и проверь ограничение `stock >= 0`.

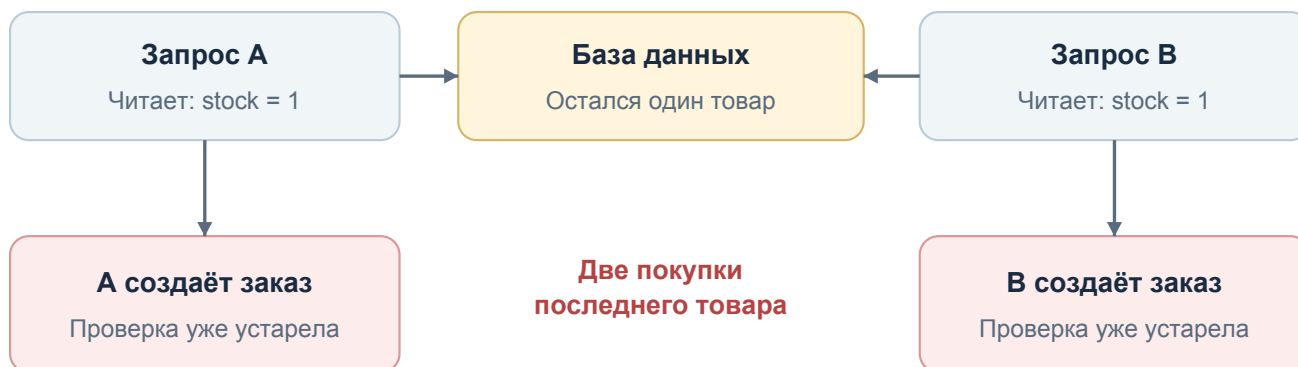
Самопроверка. Почему BEGIN и COMMIT сами по себе не устраняют все гонки?

СХЕМА 09 / К ГЛАВЕ 19

Гонка за последний товар

Сверху - уязвимая последовательность отдельных действий. Снизу - условная атомарная запись для одного товара.

А. Раздельные чтение и запись: оба увидели stock = 1



Б. Условное атомарное обновление: проходит только один



Как читать схему

При раздельном чтении оба запроса принимают решение по одному остатку. В условном UPDATE изменение разрешено только при достаточном остатке. Заказ и резерв входят в одну транзакцию. При конфликте сериализации повторяют транзакцию по правилам БД.

Проверь себя. Как проверить результат UPDATE, если RETURNING не вернул строк?

ГЛАВА 20

20. Индексы, запросы, миграции и восстановление

Индекс - дополнительная структура

Индекс позволяет БД быстрее находить подходящие строки, но занимает место и обновляется при записи. B-tree полезен для равенства, диапазонов и некоторых порядков сортировки. Составной индекс проектируют под реальные фильтры и порядок; индекс по (customer_id, created_at, id) может помочь выдаче заказов покупателя по времени.

EXPLAIN показывает план, EXPLAIN ANALYZE выполняет запрос и измеряет работу. Для изменяющихся запросов это означает реальные побочные эффекты, поэтому диагностику планируют осторожно. Сравнивай оценки количества строк с фактическими данными; большой разрыв может указывать на статистику или нетипичное распределение.

```
CREATE INDEX orders_customer_created_idx
ON orders (customer_id, created_at DESC, id DESC);

SELECT id, status, created_at
FROM orders
WHERE customer_id = 7
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

N+1 и стоимость чтения

Один запрос получает 100 заказов, затем каждый заказ отдельно загружает покупателя: возникает 101 запрос. Решением может быть JOIN, пакетная загрузка или предварительное получение нужных связей. Но огромный JOIN нескольких коллекций способен размножить строки; выбирать нужно по измерениям и объёму ответа.

Репликация создаёт копии данных для чтения или восстановления после отказа. Реплика может отставать: после записи пользователь иногда не увидит её при немедленном чтении оттуда. Для критичного read-after-write нужна явная политика. Репликация не заменяет резервные копии: ошибочное удаление может распространиться на все реплики.

Миграции без остановки

Подход expand-contract: сначала добавить совместимую структуру, затем выпустить код, понимающий оба состояния, перенести данные и только потом удалить старое. Переименование поля одним шагом способно сломать ещё работающие экземпляры старой версии. Длительные блокировки и объём заполнения нового столбца оценивают на реалистичных данных.

Резервная копия полезна, только если её удастся восстановить. RPO определяет допустимую потерю данных по времени, RTO - допустимое время восстановления. Проверяй не только наличие файла, но и процедуру: ключи, доступ, совместимость версий, полноту данных и запуск приложения после восстановления.

Типичная ошибка. Добавлять индекс на каждый столбец, не проверяя запросы и цену записи.

Практика. Сравни план одного запроса до и после индекса на тестовых данных; отдельно восстанови тестовую БД из копии.

Самопроверка. Почему откат приложения может не сработать после несовместимой миграции?

ГЛАВА 21

21. Аутентификация, авторизация и сессии

Кто ты и что тебе разрешено

Аутентификация устанавливает личность, авторизация проверяет конкретное действие. Пользователь может быть успешно авторизован в смысле входа в интерфейс, но не иметь доступа к чужому заказу; в техническом проектировании эти две проверки нужно различать. Роль admin - лишь один способ выражения прав, а не замена всем проверкам.

RBAC связывает права с ролями. ABAC принимает решение по атрибутам: организация, владелец, состояние ресурса и контекст. Для многопользовательской системы важна tenant isolation: каждый запрос и ключ кеша должны учитывать организацию там, где данные разделены.

```
SELECT id, status
FROM orders
WHERE id = $1 AND customer_id = $2;
```

Здесь \$2 берётся из проверенной сессии. Поиск только по id с последующей выдачей объекта создаёт риск доступа к чужим данным. Проверять нужно и чтение, и изменение, и экспорт, и вложенные ресурсы.

Сессии и токены

При серверной сессии браузер хранит случайный идентификатор, сервер - состояние сессии. Её удобно отозвать. При подписанном JWT сервер может проверить подпись и claims без поиска сессии, но отзыв и изменение прав требуют отдельной политики. Подписанный JWT обычно читаем: подпись не означает шифрование. Проверяются допустимый алгоритм, подпись, issuer, audience и время действия по контракту.

OAuth 2.0 описывает делегирование доступа, OpenID Connect добавляет слой идентификации. Для входа через провайдера используют предназначенный для этого протокол и библиотеку, а не собственную интерпретацию access token. Токены разных назначений нельзя взаимозаменять.

Пароли и восстановление

Пароли хранят через специализированное адаптивное хеширование с солью, например Argon2id, с параметрами под выбранную среду и актуальные рекомендации. Простого SHA-256 недостаточно. При входе учитывают ограничение попыток, безопасные сообщения, защиту сессии и при необходимости MFA. Основные рекомендации по аутентификации см. S10.

Токен восстановления должен быть непредсказуемым, ограниченным по времени и одноразовым. После смены пароля политика активных сессий должна быть определена. Секреты и токены нельзя писать в обычные логи.

Типичная ошибка. Хранить роль только в localStorage и доверять ей на сервере.

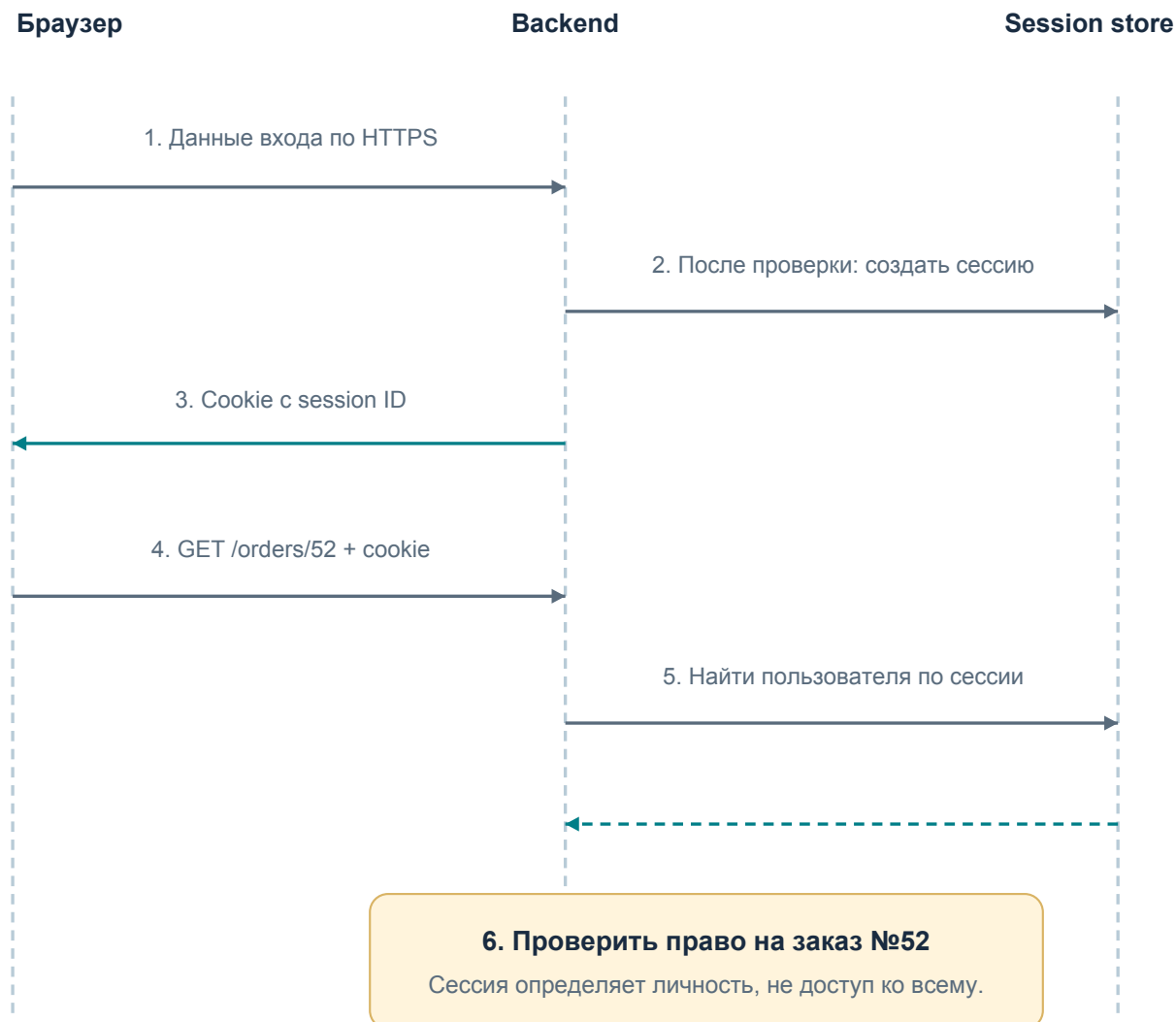
Практика. Составь матрицу «гость / покупатель / менеджер» и действий над своим и чужим заказом.

Самопроверка. Почему случайные UUID не заменяют проверку прав?

СХЕМА 10 / К ГЛАВЕ 21

Сессия: вход и следующий запрос

Упрощённая схема серверной сессии: cookie содержит идентификатор, а связанные с ним данные находятся в доверенном хранилище.



Как читать схему

Успешный вход создаёт сессию. Следующий запрос использует её для определения пользователя. Затем выполняется отдельная проверка доступа к заказу. Атрибуты cookie и CSRF-защита выбираются по архитектуре.

Проверь себя. Почему действующая сессия не разрешает читать любой orderId?

22. Безопасность входов, браузера и файлов

Инъекции и доверие

SQL-инъекция возникает, когда данные становятся частью команды. Параметризованный запрос отделяет значения от структуры SQL. Имена столбцов и направления сортировки обычно нельзя безопасно передать как обычный параметр значения; для них используют разрешённый список. Та же идея относится к командам shell, шаблонам и другим интерпретаторам.

```
// Схема для драйвера PostgreSQL:  
await db.query(  
  'SELECT id FROM users WHERE email = $1',  
  [input.email]  
);
```

XSS позволяет внедрить исполняемый код в контекст сайта. Выводи пользовательский текст как текст, используй контекстное экранирование. Если разрешён HTML, нужна надёжная очистка с явной политикой. CSP добавляет защитный слой, но не заменяет корректный вывод.

CSRF и CORS

CSRF использует то, что браузер может автоматически отправлять учётные данные, например cookie, при запросе, инициированном другим сайтом. Защита зависит от архитектуры: CSRF-токены, проверки Origin, подходящие SameSite cookies и корректная семантика методов. GET не должен выполнять бизнес-изменения.

CORS регулирует, может ли браузерный JavaScript читать определённые междоменные ответы; это не серверная авторизация и не общий запрет запросов из интернета. Злоумышленник может отправить запрос вне браузера. Для credentialed-запросов нельзя бездумно сочетать произвольное происхождение с разрешением cookies. Конкретные сценарии безопасности разобраны в S11.

SSRF и загрузка файлов

SSRF возникает, если сервер по пользовательскому URL обращается к внутренним адресам или служебным endpoint. Проверяй разрешённые назначения, перенаправления и сетевые границы; одной проверки текста URL часто мало из-за DNS и альтернативных форм адреса.

При загрузке файла нужны лимиты размера, проверка допустимого формата и безопасное имя хранения. Пользовательское имя не должно определять путь на диске. MIME-заголовок и расширение не доказывают содержимое. Не позволяй загруженному файлу исполняться как серверный код. Для приватных объектов проверяй доступ и используй ограниченные по сроку ссылки при необходимости.

Типичная ошибка. Считать «у нас есть HTTPS и CORS» полноценной защитой всех входных данных.

Практика. Найди границы доверия в форме загрузки аватара: имя, размер, байты, URL результата и право изменения профиля.

Самопроверка. Чем XSS отличается от CSRF по механизму выполнения атаки?

ГЛАВА 23

23. Типы ошибок и стратегия обработки

Несколько классификаций одновременно

Синтаксическая ошибка мешает разобрать программу, ошибка типов выявляется проверкой типов, runtime-ошибка возникает при выполнении, логическая даёт неверный результат без обязательного исключения. Эти категории описывают место обнаружения, а не способ восстановления. Неверная скидка может не вызывать ни одного exception и всё равно быть серьёзной ошибкой.

По смыслу полезно различать ожидаемый отказ и дефект программы. «Нет товара» - нормальный исход сценария, «невозможно прочитать поле у undefined» - вероятный дефект. По времени - временные и устойчивые причины. По определённости - операция точно отклонена или её результат неизвестен. Один timeout одновременно является сетевой проблемой и источником неопределённости.

Карта реакции

Некорректный ввод: вернуть ошибки полей и позволить исправить. **Нет прав:** отказать, не повторять автоматически. **Конфликт версии:** перечитать состояние и предложить осмысленное разрешение.

Ошибка сериализации БД: ограниченно повторить всю транзакцию, если сценарий допускает.

Перегрузка зависимости: применить бюджет повторов и задержку. **Ошибка программы:** сохранить контекст, безопасно завершить запрос и исправить причину.

Out of memory, исчерпание соединений и файловых дескрипторов относятся к ресурсным сбоям. Их нельзя надёжно устранить одним catch: нужно ограничивать нагрузку, устранять утечки и корректно управлять процессами. Частичный успех требует отдельного статуса: например, заказ создан, но письмо ещё не отправлено.

```
{
  "error": {
    "code": "OUT_OF_STOCK",
    "message": "Товар закончился",
    "requestId": "req_7f31",
    "details": { "productId": "p1" }
  }
}
```

Стабильный code нужен клиентской логике, message - человеку, requestId - поиску диагностики. Для внутренней ошибки не возвращай SQL, stack trace, пути и секреты. Технические подробности остаются в защищённых логах.

Где перехватывать

Перехватывай ошибку там, где можешь изменить поведение: повторить безопасную операцию, выбрать fallback, преобразовать доменный отказ в HTTP или завершить процесс корректно. Пустой catch скрывает проблему. Повторное логирование на каждом слое засоряет наблюдаемость; лучше сохранить причину и контекст и записать событие на подходящей границе.

Типичная ошибка. Все исключения превращать в 400: дефект сервера становится «виной пользователя», а мониторинг теряет реальные сбои.

Практика. Для invalid email, deadlock, HTTP 403, timeout оплаты и ошибки null укажи категорию, ответ и допустимость повтора.

Самопроверка. Чем ожидаемый отказ отличается от необработанного исключения?

СХЕМА 11 / К ГЛАВЕ 23

Как выбирать реакцию на ошибку

Сначала определи причину и возможность восстановления. Затем проверь, известен ли результат операции и допустим ли повтор.



Как читать схему

Ошибку ввода исправляют, дефект расследуют, временный сбой иногда повторяют. При неизвестном исходе платежа нужна сверка или гарантированно идемпотентный повтор. Даже безопасные повторы ограничивают по времени и числу.

Проверь себя. Почему один timeout не позволяет сделать вывод «платёж не произошёл»?

ГЛАВА 24

24. SOLID, DRY, KISS, YAGNI и чистый код

Принципы как способ рассуждать

SRP предлагает объединять код по связанной ответственности и причинам изменения. OCP побуждает строить устойчивые точки расширения, когда варианты уже понятны. LSP требует, чтобы подстановка реализации не нарушала контракт. ISP помогает не навязывать потребителю ненужные методы. DIP направляет зависимости к подходящим абстракциям вместо привязки бизнес-правил к техническим деталям.

SOLID не требует создавать интерфейс для каждого класса. Например, маленькой чистой функции расчёта суммы не нужна фабрика из пяти файлов. Польза принципа измеряется тем, стало ли легче менять, проверять и понимать конкретный код.

Знание и дублирование

DRY касается дублирования знания. Если правило максимального количества товара выражено в пяти несогласованных местах, изменения опасны. Но два похожих фрагмента для разных бизнес-процессов могут развиваться независимо. Слишком ранняя общая функция с десятком флагов делает их зависимыми без причины.

KISS предлагает выбирать понятное решение. YAGNI удерживает от реализации гипотетических возможностей. Они не означают отказ от обработки ошибок, безопасности и требований, которые уже существуют. Чистые функции помогают локально проверять расчёты; побочные эффекты удобно держать на явных границах.

```
function totalMinor(
  items: readonly { priceMinor: number; quantity: number }[]
): number {
  return items.reduce(
    (sum, item) => sum + item.priceMinor * item.quantity,
    0
  );
}
```

Это учебная функция для уже проверенных неотрицательных безопасных целых чисел. В реальном контракте нужно ограничить диапазон результата либо выбрать `decimal/BigInt` и договориться о сериализации. Отсутствие чтения БД и текущего времени делает результат воспроизводимым.

Композиция и инкапсуляция

Композиция собирает поведение из зависимостей; наследование выражает отношение подстановки и может сильнее связывать реализацию. Инкапсуляция защищает допустимые состояния через операции вместо бесконтрольного изменения полей. Название `cancelOrder` яснее, чем `process(data, true, false)`.

Рефакторинг меняет внутреннее устройство при сохранении наблюдаемого поведения. Технический долг - принятые или накопленные издержки будущих изменений; полезно фиксировать конкретный риск и условие возврата к нему, а не называть долгом любой неидеальный код.

Типичная ошибка. Применять паттерн потому, что он «правильный», без описания проблемы и цены решения.

Практика. Раздели функцию, которая читает HTTP, считает скидку, пишет БД и отправляет письмо, на понятные ответственности.

Самопроверка. Когда устранение похожего кода может ухудшить архитектуру?

ГЛАВА 25

25. Модули, Clean Architecture, Hexagonal и DI

Граница важнее папки

Модуль скрывает внутренние детали и предоставляет небольшой контракт. Высокая cohesion означает, что его части решают связанную задачу; низкая coupling - что изменения не требуют одновременно менять множество соседей. Папка orders не становится модулем, если любой код напрямую меняет её таблицы и внутренние объекты.

В слоистой архитектуре верхний уровень координирует сценарии, доменный код выражает правила, инфраструктура выполняет технические операции. Clean Architecture подчёркивает направление зависимостей к внутренним правилам. Hexagonal Architecture описывает ports - контракты взаимодействия - и adapters, подключающие HTTP, БД и другие внешние системы.

Инверсия зависимостей на примере

```
interface OrderRepository {
  findOwner(orderId: string): Promise<string | null>;
}

class OrderAccess {
  constructor(private readonly orders: OrderRepository) {}

  async canRead(orderId: string, userId: string) {
    return (await this.orders.findOwner(orderId)) === userId;
  }
}
```

Класс знает контракт, но не конкретный SQL-драйвер. В production можно передать PostgresOrderRepository, в узком тесте - контролируруемую реализацию. Для самого чтения заказа проверку и выборку лучше согласовать, чтобы между отдельными действиями не возникла смена состояния. Пример показывает DI, а не полный безопасный endpoint.

Dependency Injection означает передачу зависимости извне. Контейнер DI автоматизирует сборку объектов, но не обязателен. Composition root - место, где конкретные реализации связываются в приложение. Нужно понимать scope: singleton живёт долго, request-scoped объект относится к запросу. Нельзя случайно хранить текущего пользователя в общем singleton.

Паттерны как инструменты

Strategy заменяет алгоритм, например расчёт доставки. Adapter переводит чужой API в свой контракт. Factory создаёт объект с нужными зависимостями. Decorator добавляет поведение вокруг интерфейса, например измерение времени. Observer уведомляет подписчиков; в асинхронной системе дополнительно нужны правила доставки и ошибок.

Архитектурная абстракция полезна, если скрывает реальную изменчивость. Repository, буквально повторяющий каждый метод ORM и не добавляющий смысла, может только увеличить объём кода. Выбирай глубину разделения по сложности сценария.

Типичная ошибка. Считать внедрение зависимостей гарантией слабой связанности, хотя интерфейс всё ещё раскрывает все детали конкретного провайдера.

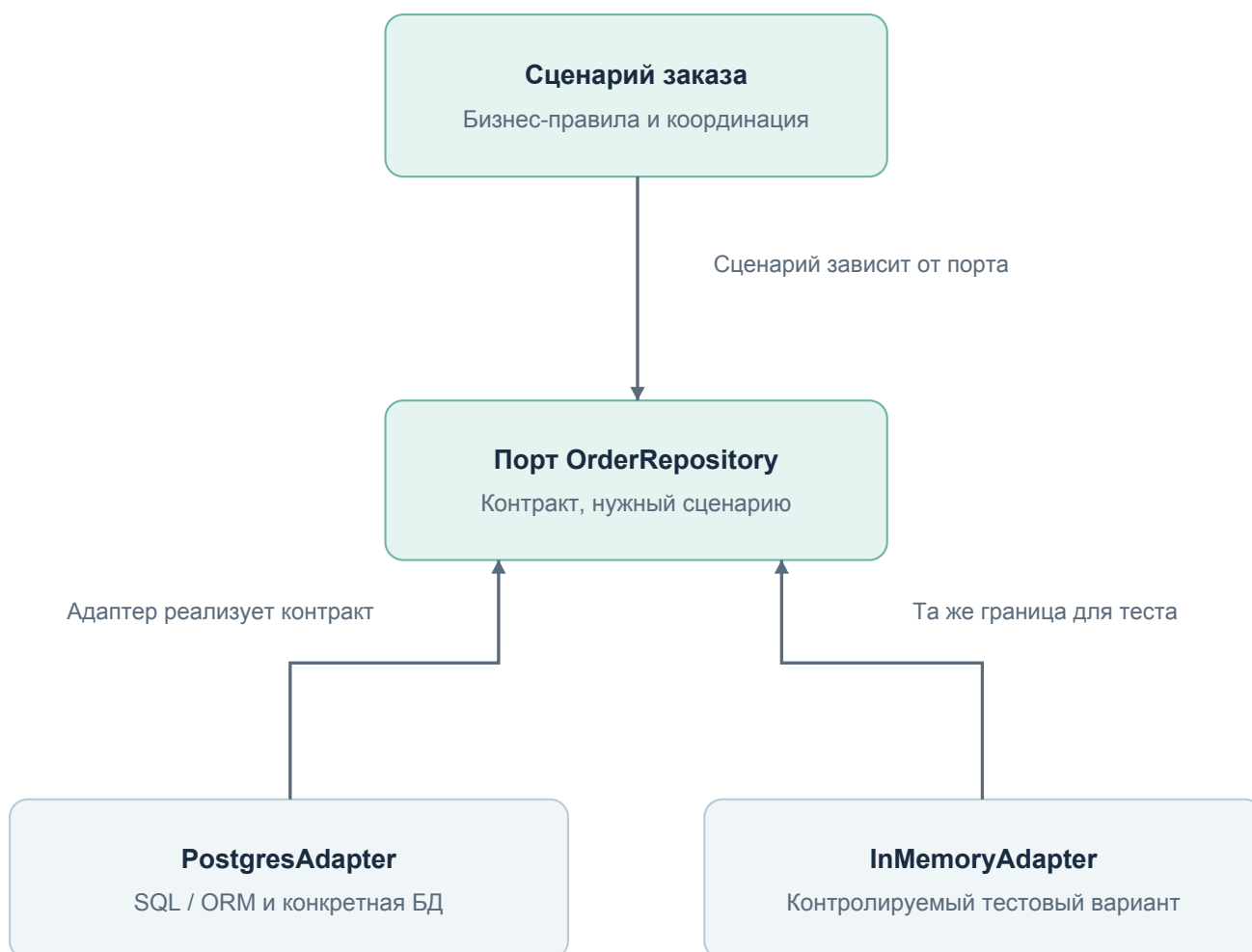
Практика. Определи порт отправки уведомлений так, чтобы бизнес-код не зависел от HTTP-формата почтового сервиса.

Самопроверка. Почему направление импортов и направление выполнения не обязаны совпадать?

СХЕМА 12 / К ГЛАВЕ 25

Зависимости вокруг бизнес-сценария

Порт описывает потребность бизнес-кода. Конкретные способы хранения подключаются через реализации этого контракта.



Стрелки показывают зависимости исходного кода. При выполнении вызов проходит через переданную реализацию. Composition root собирает объекты.

Как читать схему

Сценарий не должен импортировать конкретный SQL-драйвер, если выбран такой архитектурный подход. Тестовая реализация позволяет проверять сценарий изолированно, но SQL и ограничения всё равно проверяют интеграционными тестами.

Проверь себя. Что меняется при замене PostgreSQL-адаптера, если контракт порта остаётся прежним?

ГЛАВА 26

26. DDD, монолит, микросервисы и события

Предметная область

DDD начинается с понимания языка бизнеса. Entity сохраняет идентичность при изменении свойств: заказ остаётся тем же заказом после оплаты. Value object определяется значением: Money(1000, RUB) или адрес как снимок. Aggregate объединяет данные с общей границей согласованности; его корень контролирует допустимые изменения.

Bounded context ограничивает смысл модели. «Товар» в каталоге означает описание и категорию, на складе - остатки и расположение, в оплате - стоимость строки. Попытка создать один универсальный объект для всех контекстов часто связывает независимые изменения. DDD не требует микросервисов: контексты можно выразить модулями одного процесса.

Варианты размещения

Монолит развёртывается как единое приложение. Модульный монолит сохраняет внутренние границы и часто упрощает транзакции, отладку и выпуск. Микросервисы позволяют независимо развёртывать и масштабировать части, но добавляют сеть, частичные отказы, распределённые данные и операционные затраты. Плохо разделённые сервисы могут требовать синхронного выпуска всех компонентов и превратиться в распределённый монолит.

Модульный монолит:

```
app
  catalog | orders | payments | notifications
      одна единица развёртывания
```

Возможное выделение позже:

```
app -> очередь -> обработчик тяжёлых отчётов
```

Выделение отчётов имеет понятную причину: другой профиль нагрузки и отдельный жизненный цикл. Разделение только по техническим слоям - «сервис контроллеров» и «сервис SQL» - часто добавляет сетевые переходы без автономности.

События и согласованность

Команда выражает просьбу выполнить действие: ConfirmPayment. Событие сообщает свершившийся факт: PaymentConfirmed. Синхронный вызов удобен, когда нужен немедленный результат; асинхронный обмен отделяет время выполнения, но требует отслеживания статуса и повторов.

CQRS разделяет модели чтения и изменения там, где у них разные потребности. Это не обязательно разные сервисы или базы. Event sourcing хранит изменения как последовательность событий и восстанавливает состояние; он усложняет эволюцию схем, проекции и исправление данных, поэтому требует отдельного обоснования.

Типичная ошибка. Делать каждый класс отдельным сервисом или называть любую отправку сообщения event sourcing.

Практика. Выдели контексты магазина и укажи, кому принадлежат цена каталога, остаток, факт оплаты и адрес заказа.

Самопроверка. Какую новую проблему создаёт перенос вызова обычной функции через сеть?

ГЛАВА 27

27. Внешние API, платежи и webhooks

Чужая система - отдельная граница

У внешнего сервиса свои задержки, лимиты, ошибки и правила совместимости. Адаптер переводит его формат в понятный приложению контракт. Укажи timeout, обработку отказов, безопасные повторы и политику логирования. Не записывай токен авторизации и платёжные секреты в диагностический лог.

Неудачный ответ и потерянный ответ различаются. Если платёжный запрос завершился таймаутом, сначала выясни статус по идентификатору или повтори с тем же поддерживаемым провайдером ключом идемпотентности. Новый платёж «на всякий случай» может списать средства дважды.

Webhook как входящий API

Webhook - HTTP-уведомление от внешней системы. Источник проверяют по документированному механизму подписи, а не по одному полю sender в JSON. Некоторые подписи вычисляются по исходным байтам тела, поэтому повторная сериализация меняет проверяемое сообщение. Дополнительно учитывают время, защиту от replay и ротацию секретов по правилам провайдера.

```
Получить исходное тело и служебные заголовки
-> проверить подпись и допустимость события
-> проверить event_id на повтор
-> надёжно сохранить событие / работу
-> ответить согласно контракту провайдера
-> обработать бизнес-изменение
```

Успешный ответ нельзя отправлять раньше, чем событие надёжно принято: процесс может упасть и потерять единственную копию. Длительную работу обычно выносят в очередь. События могут приходить не по порядку; переходы статусов должны учитывать это, а при сомнении состояние сверяют с источником.

Файлы и долгие операции

Для больших загрузок часто используют объектное хранилище и ограниченную по сроку ссылку. Backend проверяет право загрузки, метаданные и завершённость, а дальнейшая обработка идёт фоном. Отчёт можно представить ресурсом /jobs/{id}: queued, running, succeeded, failed. Хранение прогресса и результата должно переживать перезапуск worker, если это требуется контрактом.

Типичная ошибка. Доверять редиректу браузера на страницу «успех» как единственному подтверждению оплаты.

Практика. Опиши обработку двух одинаковых webhook и события оплаты, пришедшего после запроса отмены.

Самопроверка. Почему IP отправителя сам по себе обычно недостаточен для подтверждения подлинности webhook?

ГЛАВА 28

28. Очереди, Outbox и Saga

Что даёт очередь

Producer публикует задачу, broker хранит или доставляет её, consumer выполняет и подтверждает. Очередь сглаживает кратковременные пики и отделяет время ответа API от длительной работы. Но если входящий поток постоянно больше производительности обработчиков, длина очереди будет расти: очередь не создаёт вычислительную мощность.

At-most-once допускает потери ради отсутствия повторной доставки. At-least-once допускает повторы ради повышения вероятности обработки. Обещание exactly-once нужно читать в точной области гарантий системы: внешний побочный эффект, например письмо, не становится автоматически однократным. Обработчик должен быть готов к повтору после падения между действием и подтверждением.

Повтор и проблемная задача

Используй ограниченное число попыток с задержкой. Если задача стабильно падает, отправь её в dead-letter queue или аналогичное состояние для анализа. Сохрани причину и исходный идентификатор. Порядок внутри очереди не гарантирует порядок завершения при нескольких consumers; при необходимости ключ партиционирования связывают с сущностью.

Outbox

Проблема dual write: БД сохранила заказ, но публикация сообщения не удалась. Или сообщение опубликовано, а транзакция откатилась. Outbox сохраняет изменение и намерение отправить событие в одной локальной транзакции. Отдельный publisher читает outbox и доставляет сообщение.

```
Транзакция БД:
INSERT order
INSERT outbox(event_id, type, payload)
COMMIT

Publisher:
прочитать outbox -> отправить -> отметить доставку
```

Если publisher упадёт после отправки до отметки, сообщение уйдёт повторно. Поэтому Outbox согласует запись с намерением публикации, но не отменяет дедупликацию на приёме.

Saga и компенсации

Saga связывает локальные транзакции нескольких участников. Например: резерв товара -> попытка оплаты -> подтверждение заказа. При отказе оплаты запускается компенсация снятия резерва. Это бизнес-действие, а не магический глобальный rollback. Возврат платежа может занять время или сам завершиться ошибкой; нужны статус, повторы и ручное разрешение крайних случаев. Паттерны распределённых сценариев см. S12.

Типичная ошибка. Считать отправку письма внутри транзакции БД откатываемой вместе с INSERT.

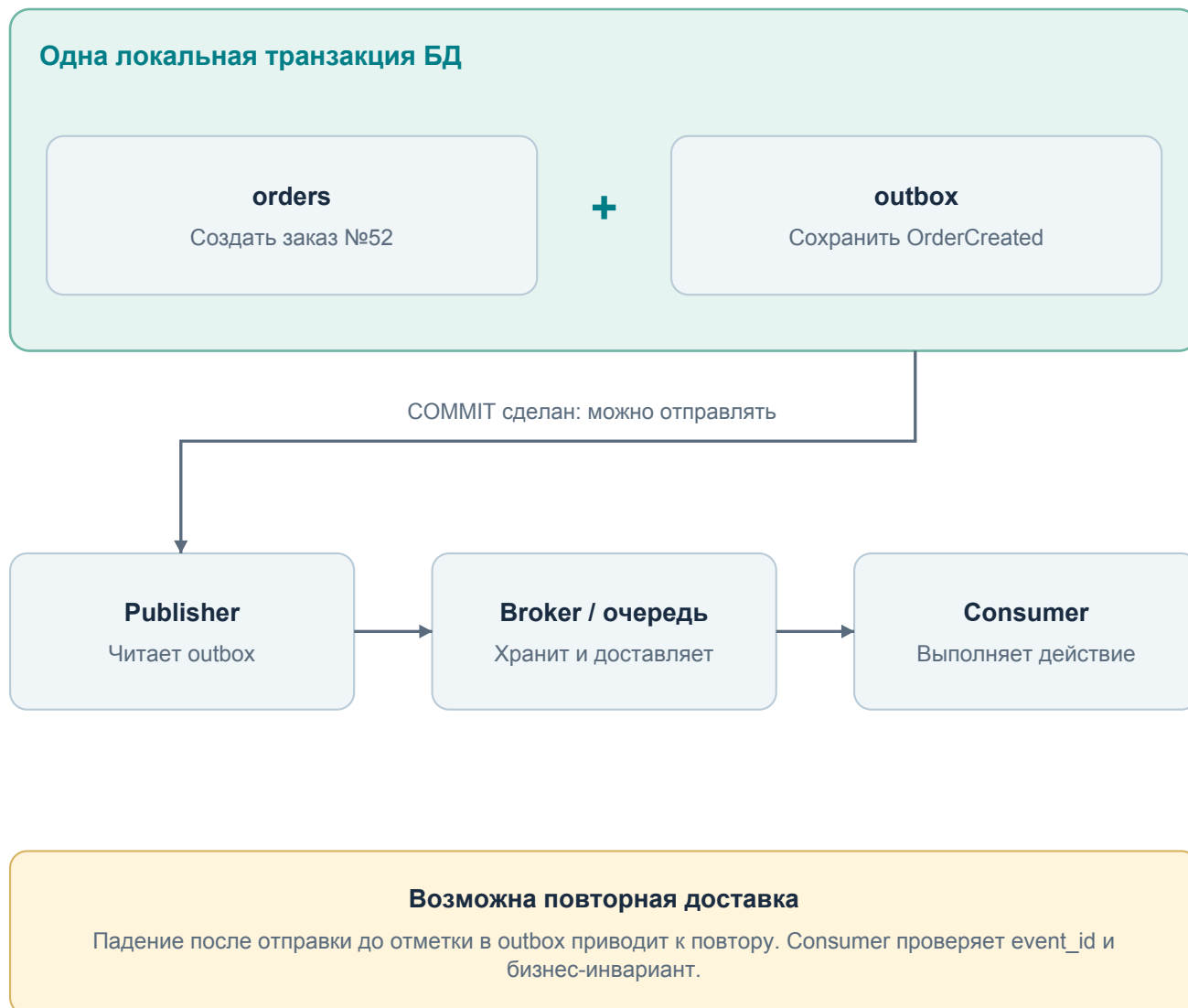
Практика. Нарисуй падение в каждой точке схемы Outbox и укажи, где возможен повтор.

Самопроверка. Почему consumer всё равно должен быть идемпотентным при использовании Outbox?

СХЕМА 13 / К ГЛАВЕ 28

Outbox: запись и доставка события

Зелёная область - общая транзакция. Всё ниже происходит после её фиксации и может потребовать повторов.



Как читать схему

Заказ и намерение отправить событие сохраняются вместе. Publisher доставляет его в брокер. Падение до отметки об отправке может породить дубликат, поэтому consumer должен корректно обрабатывать повтор.

Проверь себя. В какой точке событие может отправиться второй раз и что защищает от повторного эффекта?

ГЛАВА 29

29. Таймауты, retry, circuit breaker и backpressure

Бюджет времени

Если API должен завершиться за две секунды, три последовательных вызова по две секунды каждый не укладываются в цель. Deadline задаёт предел всей операции, timeout - ожидание конкретного шага. Бюджет передают вниз по цепочке; отмена ожидания должна по возможности прекращать ненужную работу, но уже произошедший эффект нужно учитывать отдельно.

Retry имеет смысл для временного сбоя и безопасного повторения. Exponential backoff увеличивает задержку, jitter добавляет случайность и уменьшает одновременный повтор тысяч клиентов. Число попыток, общий deadline и Retry-After ограничивают поведение. Повторы на каждом из трёх слоёв могут многократно умножить нагрузку.

Учебная политика:

- максимум 3 попытки в общем бюджете 2 секунды;
- только разрешённые временные ошибки;
- тот же idempotency key для той же операции;
- задержка со случайным разбросом;
- прекращение при отмене или исчерпании бюджета.

Конкретные числа здесь иллюстрируют контракт, а не универсальную production-настройку.

Circuit breaker и изоляция

Circuit breaker временно прекращает вызовы явно проблемной зависимости, затем разрешает ограниченные пробы восстановления. Он не исправляет сбой и не заменяет timeout. Bulkhead разделяет пулы ресурсов: зависший сервис рекомендаций не должен занять все потоки, необходимые для оформления заказа. Эти механизмы и их компромиссы представлены в каталоге S12.

Fallback должен сохранять смысл. Можно временно скрыть рекомендации или показать кешированное описание. Нельзя подставить «успешно оплачено» при недоступности платежей. Деградация считается корректной только при явно допустимом поведении.

Backpressure

Когда downstream не успевает, ограничивают конкурентность, замедляют producer, используют ограниченный буфер или отклоняют часть работы. Бесконечная очередь в памяти приводит к задержкам и исчерпанию памяти. Admission control решает, принимать ли новую работу с учётом ресурсов. На перегрузку лучше ответить предсказуемо, чем принять всё и потерять всю систему.

Типичная ошибка. Автоматически повторять 401, ошибки валидации и любую операцию с неизвестным побочным эффектом.

Практика. Рассчитай верхнюю границу времени трёх попыток с timeout 400 мс и задержками 100 и 200 мс; затем добавь общий deadline.

Самопроверка. Почему retries способны превратить частичный сбой в полный отказ?

30. Кеширование, CDN и инвалидация

Копия ради скорости

Кеш хранит производное представление, которое можно восстановить из источника истины. Он уменьшает задержку или нагрузку ценой дополнительной логики актуальности. Различай кеш браузера, CDN, память процесса и общий серверный кеш. Несколько экземпляров приложения имеют разные локальные кешы, если их явно не согласовывать.

При cache-aside приложение сначала читает кеш, при промахе обращается к источнику и сохраняет результат. TTL ограничивает срок использования, но не гарантирует, что данные всё это время актуальны. Write-through обновляет данные через слой, согласующий запись с кешем; write-behind откладывает запись и требует продумывать потери и порядок. Названия не заменяют точного контракта системы.

```
GET product:42
  hit  -> вернуть кешированное представление
  miss -> прочитать БД -> сохранить с TTL -> вернуть
```

Изменение товара:
зафиксировать БД -> инвалидировать связанную копию

Это базовая схема. При конкуренции читатель может вернуть в кеш старое значение уже после инвалидации. Для строгих требований нужны версии, согласованный порядок, дополнительные проверки или отказ от кеширования критичной части. Цена оформления заказа должна проверяться по доверенному актуальному источнику.

Лавина промахов

Cache stampede возникает, когда популярный ключ истёк и много запросов одновременно идут в БД. Помогают объединение одновременных загрузок одного ключа, распределение TTL, предварительное обновление и осторожная выдача устаревших данных там, где это допустимо. Блокировка загрузки тоже должна иметь срок и сценарий отказа.

Ключ должен включать всё, что влияет на результат: идентификатор, язык, версию и при необходимости пользователя или организацию. Пропуск tenant_id может стать утечкой данных. Отрицательный кеш для «не найдено» полезен, но не должен слишком долго скрывать недавно созданный ресурс.

CDN

CDN размещает копии ближе к пользователю и снижает нагрузку на origin. Для статических файлов удобны хешированные имена и длительное кеширование. Персональные API-ответы требуют отдельной политики. Vary сообщает о зависимости представления от заголовка, но слишком много вариантов снижает эффективность кеша.

Типичная ошибка. Добавить Redis и считать проблему скорости решённой, не измерив hit rate и стоимость промахов.

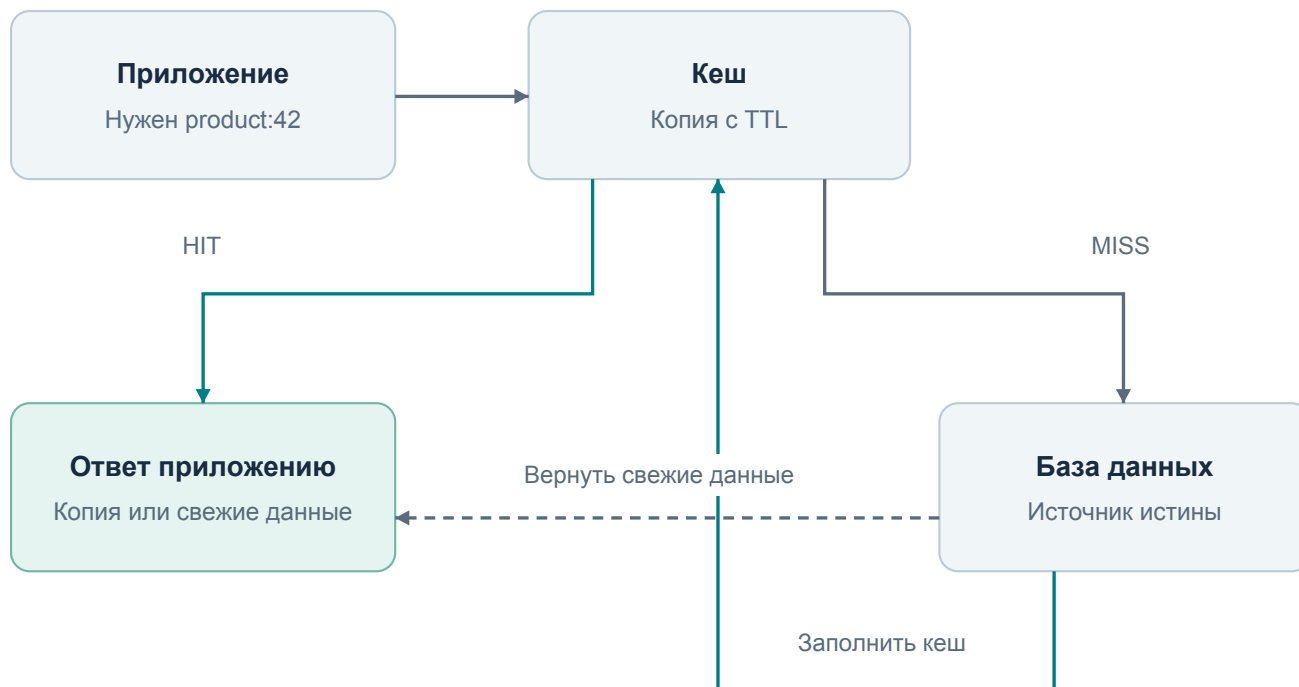
Практика. Раздели данные магазина на те, что можно устаревать минуту, секунду и нельзя использовать без проверки при покупке.

Самопроверка. Почему TTL не является гарантией бизнес-корректности?

СХЕМА 14 / К ГЛАВЕ 30

Cache-aside: попадание и промах

HIT означает найденную копию, MISS - необходимость получить данные из источника. Кеш является производным состоянием.



Изменение данных требует политики актуальности

TTL и инвалидация ограничивают устаревание, но не заменяют проверку цены и остатка при покупке.

Как читать схему

При промахе приложение читает БД, заполняет кеш и возвращает результат. При изменении данных копию обновляют или инвалидируют по выбранной политике. Конкурентность может снова занести устаревшее значение.

Проверь себя. Какие данные заказа нельзя без проверки брать из устаревшего кеша?

ГЛАВА 31

31. WebSocket, SSE и обновления в реальном времени

Три модели обновления

Polling периодически запрашивает состояние. Он прост и часто достаточен, но расходует запросы даже без изменений и даёт задержку до следующего опроса. SSE передаёт события от сервера к клиенту поверх длительного HTTP-ответа. WebSocket создаёт двусторонний обмен сообщениями и подходит для частых интерактивных событий.

Выбор зависит от направления и частоты. Для статуса отчёта может хватить polling. Для ленты серверных обновлений подходит SSE. Для совместного редактора или чата может быть полезен WebSocket. Само наличие постоянного соединения не решает хранение истории, доставку после разрыва и авторизацию сообщений.

```
// Браузерный пример SSE, сервер должен поддерживать endpoint.  
const stream = new EventSource('/api/order-events');  
stream.addEventListener('order-status', event => {  
  const update = JSON.parse(event.data);  
  console.log(update);  
});  
// При окончательном уходе из соответствующего UI:  
// stream.close();
```

Формат JSON также нужно проверять. Сервер отправляет Content-Type: text/event-stream и правильно оформленные события. Для прокси учитывают буферизацию и таймауты, иначе сообщения могут приходить пачками с задержкой.

Восстановление соединения

После разрыва клиент переподключается с задержкой. Идентификатор последнего события может позволить получить пропущенное, если сервер хранит историю и реализует продолжение. Для SSE существует механизм Last-Event-ID, но его наличие само по себе не создаёт историю на backend. Если восстановление невозможно, клиент перечитывает текущее состояние через обычный API.

Heartbeat помогает обнаруживать мёртвые соединения. Backpressure нужен и здесь: медленный клиент не должен бесконечно накапливать сообщения в памяти сервера. Важно определить, какие обновления можно объединить, а какие нужно сохранить отдельно.

Масштабирование и безопасность

Соединение привязано к обслуживающему экземпляру. Если событие возникает в другом процессе, нужна маршрутизация через общий канал или брокер. Проверь пользователя при установлении и доступ к конкретным данным; учитывай истечение сессии. Для WebSocket проверяй допустимый Origin по архитектуре и не доверяй одному факту успешного handshake.

Типичная ошибка. Использовать живое соединение как единственное место хранения состояния заказа.

Практика. Опиши поведение страницы статуса при разрыве на минуту и последующем переподключении.

Самопроверка. Почему «сообщение отправлено в сокет» не равно «пользователь надёжно получил и обработал событие»?

ГЛАВА 32

32. Тестирование, отладка и проверка контрактов

Уровни проверки

Unit-тест проверяет небольшую логику изолированно: расчёт скидки или разрешённый переход. Интеграционный проверяет взаимодействие с реальными компонентами: SQL, транзакция, сериализация. End-to-end проходит пользовательский сценарий через браузер и backend. Чем шире граница, тем больше реализма, но обычно выше стоимость запуска и диагностики.

Проверяй наблюдаемое поведение. Тест «вызван приватный метод X ровно один раз» часто привязывается к реализации и ломается после безопасного рефакторинга. Mock нужен для контроля внешней границы, но полностью замочанная БД не проверяет SQL и ограничения. Контрактные тесты помогают обнаружить несовместимость между клиентом и провайдером API.

```
// Иллюстрация для test runner с test и expect.
test('повторная отмена сохраняет cancelled', () => {
  expect(cancel('cancelled')).toBe('cancelled');
});

test('оплаченный заказ нельзя отменить этой командой', () => {
  expect(() => cancel('paid')).toThrow();
});
```

Это тесты правила из главы 15. Отдельная интеграционная проверка должна подтвердить его при двух конкурентных запросах, а не только в памяти.

Набор сценариев

Проверяй счастливый путь, границы диапазонов, отсутствие данных, неверные права, повтор, timeout и конкуренцию. Property-based testing генерирует множество входов и проверяет свойства, например неотрицательный итог. Регрессионный тест фиксирует ранее найденный дефект. Покрытие строк показывает выполненный код, но не доказывает правильность проверенных утверждений.

Отладка

Воспроизведи проблему минимальным сценарием, сформулируй гипотезу, собери данные и меняй одну существенную переменную. Stack trace указывает путь исключения, но первопричина может находиться раньше. Browser DevTools показывает сеть, DOM, память и время выполнения; серверные инструменты - CPU, память, запросы и блокировки. Линтер ищет ряд подозрительных конструкций, formatter поддерживает единый стиль.

Типичная ошибка. Исправлять нестабильный тест добавлением долгого sleep вместо ожидания конкретного состояния.

Практика. Напиши тесты на чужой заказ, последний товар и повторный idempotency key; для каждого выбери нужный уровень.

Самопроверка. Почему 100% покрытия не равно отсутствию ошибок?

ГЛАВА 33

33. Linux, Docker и управление процессом

Среда выполнения приложения

В Linux файлы и процессы связаны с пользователями и группами. Права read, write и execute определяют доступ; у каталогов execute означает возможность прохода по пути. Принцип минимальных привилегий ограничивает последствия ошибки: веб-приложению обычно не нужен root и полный доступ к диску.

Процесс получает конфигурацию, запускается, принимает запросы и завершается. Служба или supervisor управляет перезапуском и логами. SIGTERM обычно используется для запроса корректной остановки, а SIGKILL не даёт процессу выполнить очистку. При graceful shutdown приложение прекращает принимать новую работу, завершает допустимые текущие операции, закрывает соединения и выходит в пределах бюджета времени.

Сигнал остановки

- > убрать готовность к новому трафику
- > прекратить приём новых запросов / задач
- > дождаться текущих в пределах deadline
- > закрыть пулы и ресурсы
- > завершить процесс

Liveness показывает, жив ли процесс и есть ли смысл его перезапустить. Readiness - готов ли он обслуживать трафик. Не следует делать liveness зависимой от каждой внешней системы: её отказ может вызвать массовые бесполезные перезапуски.

Контейнеры

Image - шаблон файловой системы и метаданных, container - экземпляр запуска. Контейнеры обычно разделяют ядро хоста, поэтому это не полная виртуальная машина. Volume хранит данные независимо от одноразового writable-слоя контейнера. Network определяет коммуникацию, публикация порта связывает порт контейнера с хостом. Основная модель Docker описана в S13.

```
# Учебный шаблон: версия образа задаётся при сборке.
ARG NODE_BASE
FROM ${NODE_BASE}
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY dist ./dist
USER node
CMD ["node", "dist/server.js"]
```

Предполагается совместимый Node-образ с пользователем node, заранее собранный dist и отсутствие секретов в контексте. Для воспроизводимости выбирают поддерживаемый образ и фиксируют его версию или digest. Сборку TypeScript можно оформить отдельной стадией; .dockerignore исключает ненужные файлы.

Конфигурацию проверяют при запуске: отсутствие обязательного DATABASE_URL лучше обнаружить сразу. Секреты не зашивают в image и не передают через публичные frontend-переменные. Лимиты памяти и CPU требуют проверки поведения при достижении, а не только записи в конфиге.

Типичная ошибка. Хранить единственную копию базы в удаляемом слое контейнера.

Практика. Опиши, какие данные исчезнут при пересоздании контейнера приложения и какие обязаны сохраниться.

Самопроверка. Почему автоматический restart не заменяет graceful shutdown?

ГЛАВА 34

34. Reverse proxy, CI/CD и выпуск изменений

Входная точка

Reverse proxy принимает клиентские запросы и направляет их backend. Он может завершать TLS, отдавать статистику, ограничивать тело запроса и балансировать несколько экземпляров. Обычный forward proxy представляет клиента, reverse proxy - обслуживаемые серверы. Nginx описывает базовую настройку проксирования в S14.

```
# Локальный учебный фрагмент, без публичного TLS.
server {
    listen 8080;
    location /api/ {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
    }
}
```

Этот пример сохраняет /api/ в передаваемом URI при данной форме proxy_pass. Изменение завершающего следа может изменить правила пути. Для реального HTTPS нужны сертификат, ключ и отдельная конфигурация; её проверяют под конкретный домен и версию сервера. Приложение должно доверять forwarded-заголовкам только от известных прокси, иначе клиент сможет подменить важный контекст.

Путь изменения до production

CI запускает автоматические проверки коммита: установка по lock-файлу, проверка типов, линтер, тесты и сборка. CD доставляет проверенный артефакт в окружение. Лучше продвигать один и тот же собранный артефакт, чем независимо пересобирать «тот же код» с разными зависимостями. Development, staging и production отличаются данными и доступами, но должны быть достаточно похожи для полезной проверки.

Rolling постепенно заменяет экземпляры; какое-то время работают две версии. Blue-green держит два окружения и переключает трафик. Canary направляет часть трафика на новую версию и оценивает показатели. Выбор зависит от стоимости и возможности быстрого возврата.

Совместимость и откат

Rollback кода не восстанавливает автоматически удалённые данные или несовместимую схему. Поэтому миграции согласуют со старыми и новыми экземплярами. Feature flag позволяет отделить выпуск кода от включения функции, но флаг нуждается во владельце и сроке удаления; бесконечные комбинации усложняют тестирование.

Перед выпуском задают критерии успеха и остановки: ошибки, задержки, бизнес-операции. Проверка после deployment должна пройти реальный минимальный сценарий, а не только открыть главную страницу.

Типичная ошибка. Во время rolling-релиза изменить контракт так, что старая версия frontend несовместима с новым backend.

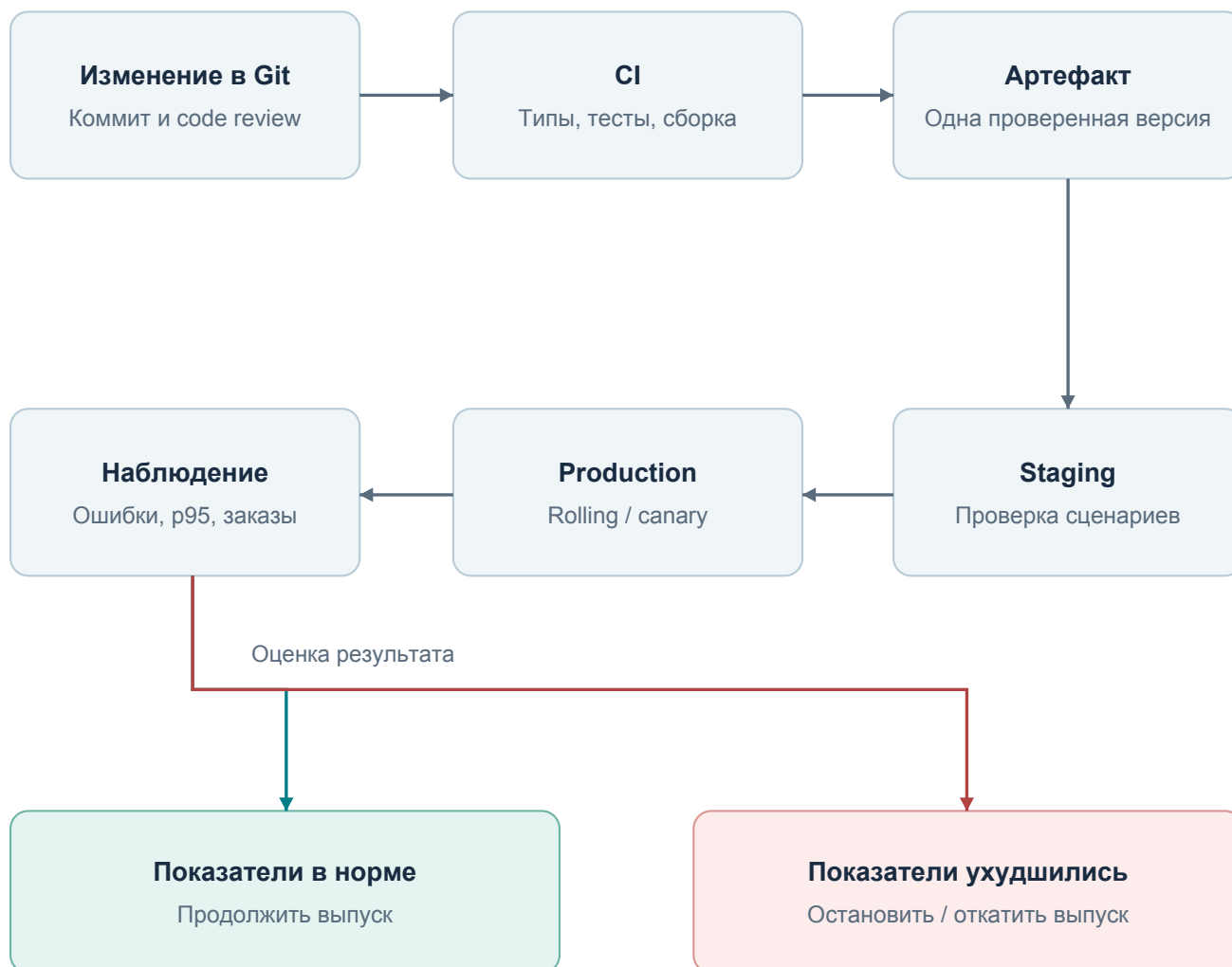
Практика. Напиши последовательность выпуска нового обязательного поля без одновременного обновления всех клиентов.

Самопроверка. Почему «сборка прошла» и «релиз работает» - разные утверждения?

СХЕМА 15 / К ГЛАВЕ 34

От коммита до работающего релиза

Проверки создают артефакт, а развёртывание переносит его в окружение. Успех оценивается по работе системы после выпуска.



Как читать схему

Одна и та же сборка проходит staging и production. Во время canary или rolling могут работать разные версии. Откат кода возможен только с учётом совместимости данных, схемы и контрактов.

Проверь себя. Что нужно проверить до rollback, если новая версия изменила схему БД?

ГЛАВА 35

35. Логи, метрики, трассировка и инциденты

Три вида сигналов

Лог описывает конкретное событие с контекстом. Метрика агрегирует измерение во времени: число запросов, размер очереди, распределение задержек. Trace связывает этапы одной операции, а span представляет отдельный участок, например обращение к БД. Эти сигналы дополняют друг друга; модель сигналов описана в S15.

Структурированные логи удобнее искать и агрегировать. Уровень error оставляют для значимых ошибок; ожидаемый отказ пользователя не обязательно требует тревоги. Correlation ID или trace ID помогает пройти путь через процессы. Не логируй пароли, полные токены и лишние персональные данные. Идентификатор пользователя в метрике может создать слишком высокую cardinality; такие детали чаще подходят логам.

```
{
  "level": "error",
  "event": "payment_timeout",
  "requestId": "req_7f31",
  "orderId": "ord_52",
  "durationMs": 900,
  "outcome": "unknown"
}
```

Поле outcome напоминает, что timeout ещё не доказывает отсутствие платежа. Лог должен поддерживать расследование, а не создавать ложную определённость.

SLI, SLO и алерты

SLI - измерение качества, например доля успешно завершённых допустимых запросов. SLO - целевое значение за период. Нужно определить числитель, знаменатель, исключения и окно. «Сайт доступен на 99,9%» без этих деталей неоднозначно. Error budget отражает допустимую долю плохих событий при выбранном SLO; это инструмент решений, а не разрешение игнорировать пользователей.

Алерт должен указывать на состояние, требующее действия. Порог CPU полезен не всегда: вычислительный worker может штатно использовать ядра полностью. Рост ошибок оформления заказа, задержки очереди или исчерпание пула непосредственно связан с функцией. Для алерта нужен runbook: где смотреть, какие гипотезы проверить, как безопасно снизить последствия.

Работа с инцидентом

Сначала оцени масштаб и уменьши ущерб: откат, ограничение трафика, отключение проблемной функции. Затем восстанови ход событий и первопричины. Разбор без поиска виноватого фиксирует технические и организационные условия, действия и конкретные улучшения. Время, версии и идентификаторы помогают соединить наблюдения.

Типичная ошибка. Создавать уведомление на каждый exception без приоритета и инструкции: команда перестаёт замечать важное.

Практика. Для «заказы стали медленнее» назови одну метрику, один полезный лог и один span, которые помогут найти причину.

Самопроверка. Чем метрика отличается от лога, даже если обе содержат длительность запроса?

ГЛАВА 36

36. Производительность и масштабирование

Что измерять

Latency - время одной операции, throughput - число завершённых операций за единицу времени.

Среднее скрывает хвост: p95 означает, что 95% наблюдений не превышают это значение, а оставшиеся могут быть значительно медленнее. Важно фиксировать окно и нагрузку: p95 при одном запросе в секунду мало говорит о поведении при тысяче.

Профилирование показывает, где расходуются CPU и память. Нагрузочное тестирование проверяет ожидаемый режим, стрессовое ищет предел и характер отказа, длительное выявляет утечки и накопление. Тестовый сценарий должен отражать реальные действия, размеры данных, кеш и долю операций, а не только быстрый /health.

Очереди и пределы

Когда система приближается к пределу ресурса, ожидание часто растёт нелинейно. При устойчивом режиме среднее количество операций в системе связано с интенсивностью поступления и средним временем пребывания: $L = \lambda \times W$. Например, 100 запросов в секунду при среднем времени 0,2 секунды дают около 20 запросов в системе в среднем. Это ориентир при выполнении условий устойчивости, не формула для пикового лимита.

Симптом: API отвечает 1,8 с

Разложение trace:

очередь пула БД	1,4 с
SQL	0,2 с
остальное	0,2 с

Вывод: сначала исследовать пул, транзакции и нагрузку, а не переписывать сериализацию JSON.

Варианты масштабирования

Вертикальное увеличивает ресурсы экземпляра. Горизонтальное добавляет экземпляры; оно требует учитывать общее состояние, пулы, фоновые задачи и балансировку. Stateless HTTP-сервер не хранит необходимое межзапросное состояние только в своей памяти; данные могут жить в БД или общем session store. Это облегчает замену экземпляра, но не делает систему полностью «без состояния».

Балансировщик распределяет запросы и исключает неготовые узлы. Алгоритм и sticky sessions влияют на распределение. Репликация чтения, партиционирование и sharding решают разные проблемы данных и добавляют сложность. Сначала найди bottleneck; увеличение числа backend-процессов не ускорит одну перегруженную таблицу автоматически.

Типичная ошибка. Проводить нагрузочный тест без ограничения стоимости и без понимания, что тест обращается к реальным платным внешним сервисам.

Практика. Составь тест: ожидаемая нагрузка, доля покупок, объём каталога, p95, допустимые ошибки и условия остановки.

Самопроверка. Почему удвоение числа серверов не гарантирует удвоение пропускной способности?

37. Командные правила и архитектурные решения

Договорённости должны быть проверяемыми

Команде нужны единые правила именования, форматирования, структуры изменений и проверки. То, что можно автоматически проверить, лучше перенести в линтер или CI. Code review посвящают корректности, понятности, безопасности, совместимости и достаточной проверке, а не спору о пробелах.

Хороший pull request описывает проблему, новое поведение и способ проверки. Небольшое связанное изменение легче понять и откатить. В ревью различай обязательное исправление, вопрос и необязательное предложение. Если требуется серьёзное архитектурное решение, его полезно обсудить до написания большого объёма кода.

ADR

Architecture Decision Record фиксирует контекст, варианты, принятое решение и последствия. Он объясняет, почему команда предпочла конкретный компромисс в тот момент. ADR не обязан быть длинным и может быть заменён новым при изменении условий.

Решение: начинать с модульного монолита.
Контекст: одна команда, общая БД, небольшой трафик.
Альтернатива: отдельные сервисы orders и payments.
Причина: важнее простота выпуска и локальные транзакции.
Последствие: независимого масштабирования модулей пока нет.
Пересмотр: при измеренной потребности в отдельном ресурсе или независимом цикле выпуска.

Документация должна помогать следующему действию. README объясняет запуск и тесты; схема показывает границы; gupbook - восстановление; OpenAPI - протокол. Если инструкция перестала соответствовать коду, её ценность быстро падает, поэтому изменения документации включают в обычный процесс.

Definition of Done

Для конкретного изменения заранее определяют, что означает готовность: сценарий работает, права проверены, миграция совместима, важные тесты проходят, ошибки наблюдаемы. Набор зависит от риска: исправление подписи кнопки не требует того же процесса, что изменение платежей.

Технический долг описывай через последствия: «долгая транзакция блокирует оформление» полезнее, чем «некрасивый код». Приоритет определяется риском и стоимостью будущих изменений. Владелец и критерий завершения делают задачу выполнимой.

Типичная ошибка. Писать правила, которые никто не применяет, и считать наличие документа доказательством качества.

Практика. Напиши ADR выбора сессий или JWT для учебного магазина, учитывая отзыв доступа и сложность сопровождения.

Самопроверка. Как понять, что архитектурное решение пора пересмотреть?

ГЛАВА 38

38. Сквозной разбор: безопасное создание заказа

Сначала контракт

Вход содержит productId и quantity, но не доверенную цену, роль или customerId. Пользователь определяется проверенной сессией. У запроса есть ключ идемпотентности. Успех возвращает идентификатор, итог и статус pending. Отсутствие товара - ожидаемый конфликт; неизвестный результат оплаты - отдельное состояние, а не доказанный отказ.

Учебные ограничения: один склад, одна валюта в заказе, резервирование до оплаты, отдельный worker уведомлений. Они нужны, чтобы не смешать базовый сценарий с частичными отгрузками и несколькими валютами.

Серверная последовательность

HTTP boundary:

проверить сессию, CSRF по модели приложения,
схему, ограничения количества и idempotency key

Транзакция БД:

занять ключ с уникальным ограничением
получить цены и атомарно зарезервировать остатки
создать заказ и снимки строк
создать outbox-событие OrderCreated
сохранить результат идемпотентной операции
commit

После фиксации:

вернуть 201 с данными заказа
запустить оплату отдельным согласованным сценарием
доставить событие уведомления через outbox

При нескольких товарах задай устойчивый порядок блокировок и откатывай всю транзакцию при нехватке любого товара. Количество одинаковых productId сначала агрегируют или явно запрещают дубликаты, чтобы ограничения не обходились разбиением строк. Для цены и скидки нужна согласованная политика чтения в той же операции.

Разбор сбоев

Ответ потерялся после commit. Повтор с тем же ключом получает сохранённый результат. **База отклонила транзакцию.** Частичный заказ не остаётся; допустимые повторы выполняются целиком. **Письмо не ушло.** Заказ остаётся созданным, worker повторяет уведомление. **Оплата пришла дважды.** event_id и правило перехода препятствуют повторному эффекту. **Оплата и отмена столкнулись.** Условные атомарные переходы и отдельная политика компенсации определяют допустимый исход.

Что проверять

Unit-тесты покрывают расчёты и переходы. Интеграционные - последний товар, rollback, уникальный ключ и доступ владельца. E2E - оформление, отображение ошибки и повтор после неопределённого ответа. Нагрузочный тест показывает пределы пула и блокировок. Логи и trace связывают requestId, orderId и eventId без раскрытия секретов.

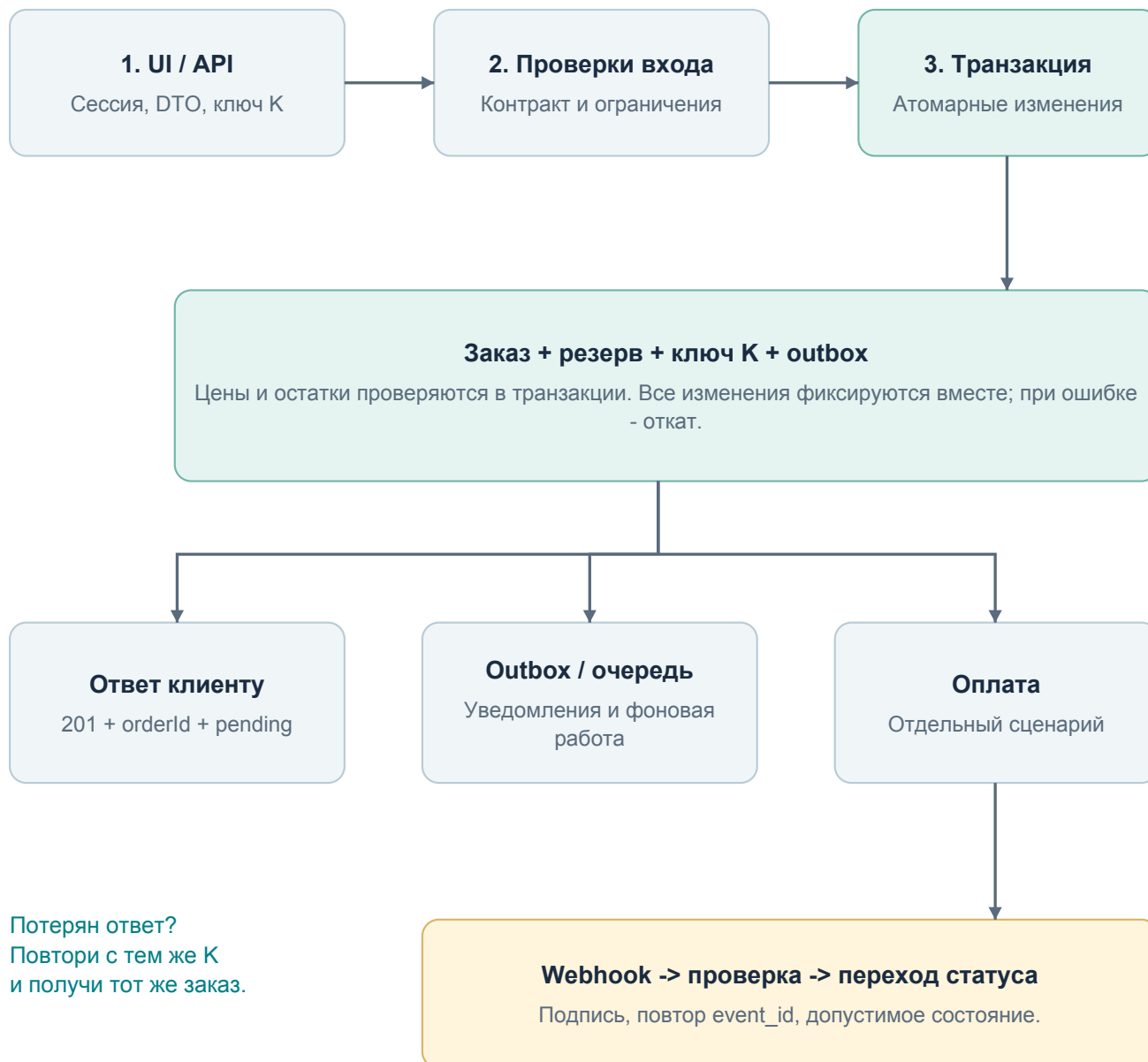
Практика. Реализуй сначала сценарий без реального платежного провайдера: адаптер должен уметь вернуть успех, отказ и timeout. Потом проверь каждую ветку.

Самопроверка. Какие действия должны быть в одной транзакции, а какие требуют отдельного механизма согласования?

СХЕМА 16 / К ГЛАВЕ 38

Оформление заказа как единый сценарий

Схема соединяет интерфейс, бизнес-правила, транзакцию и последующие внешние действия.



Потерян ответ?
Повтори с тем же К
и получи тот же заказ.

Как читать схему

Локальная транзакция согласует заказ, резерв, результат ключа и outbox. Она не откатывает внешнюю оплату. Подтверждение платежа проверяют отдельно и применяют только допустимый переход состояния.

Проверь себя. Объясни судьбу заказа, если транзакция прошла, а сервис уведомлений временно недоступен.

ГЛАВА 39

39. Учебный маршрут и итоговый проект

Этап 1. Статический интерфейс

Изучи главы 1-3 и 7. Сделай каталог, карточку товара и форму заказа с адаптивной вёрсткой. Результат: страницу можно использовать клавиатурой, длинные названия не ломают сетку, Git содержит понятные коммиты. Если этот этап сложен, не добавляй пока сервер: сначала сделай интерфейс устойчивым.

Этап 2. Интерактивность

Пройди главы 8-11. Добавь корзину, расчёт отображаемой суммы, маршруты и состояния загрузки на тестовых данных. Результат: компоненты имеют ясные входы, ошибки видимы, устаревший поиск не заменяет свежие результаты. Клиентская сумма пока только отображение, не источник истины будущего заказа.

Этап 3. API и база

Пройди главы 12-20. Выбери один backend-фреймворк, создай схему, миграции, каталог и создание заказа. Результат: данные переживают перезапуск, вход валидируется, стоимость определяется сервером, последний товар нельзя продать дважды, повтор с тем же ключом не создаёт новый заказ.

Этап 4. Доступ и ошибки

Пройди главы 21-23 и 32. Добавь сессии, выход, доступ только к собственным заказам и единый формат ошибок. Результат: прямой запрос к чужому id отклоняется, пароли не лежат открытым текстом, внутренние ошибки не раскрывают секреты, важные случаи проверены тестами.

Этап 5. Архитектура и фоновые сценарии

Пройди главы 24-31. Раздели модули, вынеси уведомления в фон, реализуй outbox или обоснуй более простой надёжный вариант под ограничения. Добавь имитатор платежей и повтор webhook. Результат: понятно, кому принадлежат данные, как восстанавливается процесс после падения и где возможна временная несогласованность.

Этап 6. Выпуск и эксплуатация

Пройди главы 4-6 и 33-37. Подключись по SSH к своей среде, настрой домен и HTTPS, автоматическую сборку, логи и резервную копию. Результат: приложение доступно по ссылке, корректно перезапускается, восстанавливается из копии и позволяет расследовать ошибку по requestId. Не используй реальные платежи для учебных проверок: достаточно sandbox провайдера или адаптера-имитатора.

Итоговая защита проекта

Покажи пять демонстраций: обычная покупка, отказ при нехватке товара, запрещённый доступ к чужому заказу, безопасный повтор запроса и восстановление после остановки worker. Для каждой объясни путь через UI, API, правила и хранилище. Затем проведи небольшой нагрузочный тест и назови измеренное узкое место.

Сроки зависят от исходной подготовки и времени на практику. Переходи по критериям результата, а не только по числу прочитанных страниц. Учебный проект полезен, когда ты можешь изменить требование и объяснить последствия, а не только показать однажды запустившийся код.

Приложение А. Ответы для самопроверки

- 01. Отключённая кнопка.** Запрос можно отправить напрямую или изменить интерфейс; право проверяется на сервере для конкретного пользователя и объекта.
- 02. Мар и UNIQUE.** Мар ускоряет локальный поиск в одном процессе. UNIQUE защищает данные от конфликтующих записей всех клиентов БД в рамках ограничения.
- 03. Локальный запуск и CI.** Могут отличаться версии runtime, зависимости, конфигурация, ОС, права и незакомпилированные файлы. Воспроизводимая сборка уменьшает эти различия.
- 04. Открытый порт.** Он подтверждает только ограниченный сетевой факт. За портом может быть неверный сервис, сломанная зависимость или отказ конкретной функции.
- 05. TLS и SSH.** TLS - защищённый транспорт для прикладных протоколов, SSH - отдельный протокол удалённого доступа, передачи файлов и туннелей со своей моделью аутентификации.
- 06. no-cache и no-store.** Первый допускает хранение с проверкой перед использованием, второй запрещает хранить соответствующее сообщение в кеше.
- 07. Ссылка и кнопка.** Ссылка перемещает к ресурсу или адресу; кнопка выполняет действие. Семантика определяет и ожидаемое поведение клавиатуры.
- 08. async и CPU.** async меняет работу с результатом и await, но не переносит синхронный цикл на другой поток.
- 09. Проверка собственного API.** Версии сторон могут разойтись, backend может иметь дефект, посредник - вернуть другое тело. Типы клиента не проверяют сетевые байты.
- 10. Пусто и не загружено.** Пусто - успешный результат без элементов; не загружено - результат ещё неизвестен. Пользователю нужны разные состояния.
- 11. Гидратация.** Серверный HTML может отображаться раньше, чем клиентский код загрузится и подключит интерактивность.
- 12. Ответственность разработчика.** Бизнес-правила, границы доступа, целостность данных, контракты ошибок, совместимость и восстановление после сбоев остаются задачами приложения.
- 13. Низкий CPU и задержка.** Процесс может ждать сеть, блокировку, диск или свободное соединение. Малая вычислительная нагрузка не означает отсутствие bottleneck.
- 14. JWT и владелец.** Проверенный токен говорит о личности и claims, но не доказывает право на произвольный объект. Нужна объектная авторизация.
- 15. Откат платежа.** Внешняя система уже могла выполнить действие. Локальный rollback не управляет её состоянием; нужны сверка и предусмотренная компенсация.
- 16. HTTP 202.** Клиенту нужны идентификатор или URL статуса и контракт состояний, ошибок и результата, чтобы узнать исход принятой работы.
- 17. Повтор POST.** При потерянном ответе действие могло завершиться. Без идемпотентности повтор способен создать второй заказ или платёж.
- 18. Историческая цена.** Она относится к совершённой покупке и должна сохраняться при последующих изменениях каталога.
- 19. Границы транзакции.** Допустимые конкурентные истории зависят от уровня изоляции, запросов и ограничений. BEGIN не делает любой код сериализуемым.
- 20. Откат после миграции.** Старый код может ожидать уже удалённое поле или прежний формат. Поэтому схема и код должны сохранять совместимость в период перехода.
- 21. UUID и доступ.** Непредсказуемый идентификатор затрудняет угадывание, но не запрещает использование случайно раскрытого чужого идентификатора.

- 22. XSS и CSRF.** При XSS чужой код выполняется в контексте доверенного сайта. При CSRF чужой источник провоцирует запрос с автоматически приложенными полномочиями пользователя.
- 23. Отказ и исключение.** Ожидаемый отказ является предусмотренным исходом бизнес-сценария. Необработанное исключение означает, что текущая граница не превратила сбой в контролируемое поведение; причиной может быть дефект или неучтённая инфраструктурная проблема.
- 24. Избыточная общая абстракция.** Похожие фрагменты могут представлять разные знания. Их объединение связывает независимые изменения и рождает сложные условные ветки.
- 25. Импорты и вызовы.** Бизнес-код может вызывать интерфейс, реализация которого расположена во внешнем слое. Зависимость исходного кода направлена к контракту, а вызов во время работы - к переданному объекту.
- 26. Сеть вместо функции.** Появляются задержки, сериализация, несовместимость версий, независимые отказы и неизвестный исход после потери ответа.
- 27. IP webhook.** Адреса меняются и могут принадлежать общей инфраструктуре; они не подтверждают целостность тела и не защищают от повторной доставки. Нужен документированный механизм подлинности.
- 28. Outbox и повтор.** Publisher может отправить событие и упасть до отметки, что отправка завершена. Следующий запуск повторит доставку.
- 29. Усиление сбоя.** Повторы повышают нагрузку на уже перегруженную зависимость, занимают пулы и могут синхронно приходить от многих клиентов.
- 30. TTL.** Данные могут измениться сразу после заполнения кеша. В течение TTL копия допустима по технической политике, но может быть непригодна для конкретного бизнес-решения.
- 31. Сокет и доставка.** Между отправкой и обработкой возможны разрыв, падение клиента и потеря состояния. Нужны подходящие подтверждения, идентификаторы и восстановление.
- 32. Покрытие.** Выполнение строки не означает, что проверены все значения, ветки, гонки и правильный результат. Можно покрыть код слабым утверждением.
- 33. Restart.** Он запускает процесс заново, но не завершает аккуратно текущие запросы и не гарантирует сохранение незавершённой работы.
- 34. Сборка и релиз.** Сборка проверяет создание артефакта. В окружении дополнительно нужны правильная конфигурация, доступность зависимостей, совместимые данные и рабочие сценарии.
- 35. Метрика и лог.** Метрика агрегирует множество измерений по определённым признакам; лог сохраняет отдельное событие с контекстом. Они отвечают на разные вопросы.
- 36. Масштабирование.** Общая БД, блокировки, внешние лимиты и последовательные участки могут оставаться прежним ограничением при добавлении экземпляров.
- 37. Пересмотр ADR.** Изменились исходные ограничения, появились измеренные проблемы или цена выбранного компромисса стала выше альтернативы.
- 38. Граница согласования.** Резерв, заказ, ключ идемпотентности и outbox можно согласовать одной транзакцией общей БД. Внешний платёж и доставка сообщения требуют своих механизмов идемпотентности, повторов и сверки.

Приложение В. Короткий словарь

API - контракт взаимодействия программ. **Endpoint** - конкретная точка обращения API. **DTO** - форма данных для передачи через границу. **Schema** - описание допустимой структуры.

Runtime - среда выполнения программы. **Process** - экземпляр выполняемой программы с ресурсами ОС. **Thread** - поток исполнения внутри процесса. **Event loop** - механизм организации событий и callback.

Domain - предметная область. **Invariant** - условие, которое система обязана сохранять. **Entity** - объект с устойчивой идентичностью. **Value object** - объект, определяемый значением.

Transaction - единица согласованного изменения в пределах гарантий хранилища. **Isolation** - правила взаимодействия конкурентных транзакций. **Deadlock** - цикл взаимного ожидания. **Race condition** - зависимость результата от нежелательного порядка конкурентных действий.

Idempotency - сохранение предполагаемого эффекта при повторении операции. **Deduplication** - распознавание и исключение повторов. **Retry** - повторная попытка. **Deadline** - предел времени всей работы.

Cache - восстанавливаемая копия для ускорения. **TTL** - время допустимого хранения или использования по контракту. **Invalidation** - признание копии непригодной. **CDN** - сеть распределённой доставки контента.

Authentication - установление личности. **Authorization** - проверка разрешения. **Session** - связь запросов с состоянием взаимодействия. **Secret** - чувствительное значение, которое нельзя раскрывать посторонним.

Deploy - доставка и запуск версии в окружении. **Rollback** - возврат к предыдущему состоянию в пределах предусмотренного механизма. **Migration** - переход схемы или данных. **Artifact** - результат сборки, например контейнерный image.

Observability - возможность понимать состояние по доступным сигналам. **Trace** - путь операции через этапы. **Span** - один участок trace. **SLO** - цель качества обслуживания.

Latency - время операции. **Throughput** - объём завершённой работы за время. **Backpressure** - ограничение потока при нехватке способности обработать его. **Bottleneck** - ресурс или этап, ограничивающий систему.

RPO - допустимая потеря данных по времени. **RTO** - целевое время восстановления. **Runbook** - инструкция действий для эксплуатации. **ADR** - запись архитектурного решения с контекстом и последствиями.

Приложение С. Официальные источники

Текст методички написан как самостоятельное объяснение со сквозными учебными примерами. Ниже - первоисточники для проверки терминов и дальнейшего углубления. Они не заменяют практику. Документация инструментов меняется: выбирай раздел для установленной версии. Короткие обозначения S1-S16 связывают главы со списком. Дополнительные страницы OWASP уточняют отдельные виды защиты.

S1. HTTP. [MDN: Overview of HTTP](#). Модель запросов и ответов. Для точной семантики методов и статусов: [IETF RFC 9110: HTTP Semantics](#). Главы 1, 4, 6, 16-17.

S2. TLS. [IETF RFC 8446: TLS 1.3](#). Базовое описание протокола; на странице RFC проверяй ссылки на обновления и статус документа. Глава 5.

S3. SSH. [OpenBSD: руководство ssh](#). Аутентификация, ключи и перенаправление портов. Глава 5.

S4. TypeScript. [TypeScript Handbook](#). Типы, сужение и generics. Глава 9.

S5. React. [React: Quick Start](#). Компоненты, props, состояние и события. Глава 10.

S6. Node.js. [The Node.js Event Loop](#). Устройство цикла событий. Главы 8 и 13.

S7. Django. [Django at a glance](#). Обзор возможностей на примере документации ветки 5.2; это не утверждение о самой новой версии. Глава 12.

S8. FastAPI. [Официальная документация FastAPI](#). API, модели и проверка данных. Глава 12.

S9. PostgreSQL. [Transaction Isolation](#). Гарантии уровней изоляции в PostgreSQL. Главы 18-20.

S10. Аутентификация. [OWASP: Authentication Cheat Sheet](#). Рекомендации по входу и управлению рисками. [OWASP: Password Storage](#). Хранение паролей и параметры алгоритмов. Глава 21.

S11. Безопасность. [OWASP Cheat Sheet Series](#). Навигация по тематическим рекомендациям. Для главы 22 используй отдельные страницы: [CSRF Prevention](#), [XSS Prevention](#), [SSRF Prevention](#), [File Upload](#).

S12. Архитектурные паттерны. [Microsoft: Cloud Design Patterns](#). Каталог задач, решений и компромиссов, включая устойчивость и распределённое взаимодействие. Главы 25-30.

S13. Docker. [What is Docker?](#). Images, containers и базовая модель платформы. Глава 33.

S14. Nginx. [Beginner's Guide](#). Структура настройки и reverse проху. Глава 34.

S15. OpenTelemetry. [Signals](#). Сигналы наблюдаемости и их назначение. Глава 35.

S16. NestJS. [Controllers](#). Маршруты и обработка HTTP-запросов в конкретном фреймворке. Главы 12 и 14.

Как читать документацию после главы

Сначала сформулируй один вопрос, например «что происходит при конфликте на Serializable?». Найди раздел первоисточника, воспроизведи маленький пример и запиши вывод с условиями. Разделяй общую идею и поведение конкретного продукта: гарантии изоляции одной СУБД нельзя автоматически переносить на другую, а устройство одного фреймворка - на весь веб.